FastReDD: Accelerating Error-Aware Relational Queries over Unstructured Data

Kaiwen Chen
University Of Toronto

Daren Chao
University Of Toronto

Ruiyu Wang
University Of Toronto

Nick Koudas
University Of Toronto

ABSTRACT

Answering analytical queries over unstructured text requires extracting relational data with strict correctness guarantees. However, exhaustive LLM extraction is prohibitively expensive , and best-effort retrieval (e.g., top-k RAG) drops long-tail evidence, silently breaking aggregate results.

We present FastReDD, a system that accelerates error-aware relational queries over unstructured data by strictly adhering to a user-defined error budget. To eliminate linear corpus scans, FastReDD introduces Adaptive Schema Discovery, which uses statistical estimators to sub-linearize schema induction. For data materialization, FastReDD employs Cascaded Conformal Guards (CCG). Using lightweight surrogate models calibrated via Conformal Risk Control, CCG performs approximate predicate pushdown to safely prune irrelevant documents prior to expensive LLM extraction. Extensive evaluations demonstrate that FastReDD reduces frontier LLM token costs by up to 10x while strictly maintaining user-defined extraction recall.

1 INTRODUCTION

In modern enterprises, the most critical evidence for operational and strategic decisions—clinical narratives, incident postmortems, financial contracts, and supply-chain communications—resides in unstructured, free-form text. Increasingly, data analysts and domain experts want to interact with this unstructured evidence exactly as they interact with traditional structured databases: by issuing analytical queries involving filters, joins, and aggregations, and expecting clear semantics with reproducible results. For example, a medical researcher might ask, "What is the average treatment cost for patients diagnosed with ground-glass opacity in New York hospitals during January 2024?".

Answering analytical queries over unstructured text at scale fundamentally diverges from standard question-answering or best-effort retrieval. In analytical workloads, failing to extract relevant evidence is not just a degradation in answer quality; it is a correctness failure. Dropping the "long tail" of evidence can silently skew counts and averages, leading to disastrous downstream business or clinical decisions.

Large Language Models (LLMs) possess the semantic reasoning capabilities required to induce schemas and extract structured tuples from messy text, acting as powerful relational extractors. However, employing a frontier LLM as an exhaustive "Oracle" across every document chunk ensures high accuracy but scales linearly, resulting in an $O(|D|)$ cost bottleneck that is prohibitively expensive for real-world corpora. Conversely, relying on standard retrieval (e.g., top-k RAG) is fast but inherently biased, offering no statistical bounds on the missed evidence. This leaves system designers with an uncomfortable tradeoff: scale the system and abandon correctness guarantees, or preserve the guarantees and abandon scalability.

FastReDD is designed to break this tradeoff by treating the expensive frontier LLM as a scarce, highly guarded resource, invoking it only where it statistically buys correctness. We frame query execution over unstructured data as an error-bounded optimization problem. FastReDD allows users to define a strict error budget (e.g., "I can tolerate at most a 5% recall loss"), and the system automatically synthesizes an execution plan to meet that budget while minimizing computational cost.

To achieve this, FastReDD introduces a decoupled, risk-aware execution architecture that adapts classic database optimization techniques to the unstructured, generative domain:

- **Adaptive Schema Discovery:** Instead of forcing users to guess schemas upfront or scanning the entire corpus, FastReDD uses adaptive sampling governed by Good-Turing estimators and Jaccard distance to structurally bound the schema.
- **Conformal Physical Operators:** We introduce the Conformal Semantic Filter (CSF) and Cascaded Conformal Guards (CCG). These operators push down relational predicates and join conditions directly to the unstructured text using lightweight surrogate models.
- **Approximate Predicate Pushdown and SIP:** FastReDD dynamically applies Sideways Information Passing (SIP) via Dictionary Guards to drop orphaned records prior to expensive extraction, drastically pruning the search space.
- **Cost-Based Joint Risk Allocation:** To minimize total query cost, FastReDD utilizes a greedy optimization algorithm to distribute the user's global error budget across the different physical operators, ensuring that budget is spent on the most effective pruning stages.

In summary, we make the following contributions:

- We formalize the optimization problem of error-bounded extraction and identify the limitations of current schema-first and brute-force approaches.
- We propose Adaptive Schema Discovery to sub-linearize the logical phase of schema induction using entropy and unseen-species estimation.
- We introduce Conformal Chunk Filtering and Cascaded Conformal Guards, two physical operators that apply Conformal Risk Control to prune data and route simple samples to cheaper models without sacrificing statistical guarantees.
- We provide an extensive evaluation demonstrating that FastReDD reduces expensive generative token costs by up to 10x while strictly adhering to user-defined extraction error bounds.

2 QUERY FRAMEWORK AND OPTIMIZATION OBJECTIVE

We formulate the problem as an ad-hoc query over a Latent Relational structure in a schema-agnostic setting, where both the schema and the corresponding relational data must be induced from the corpus to satisfy an analytic intent.

2.1 The Unstructured Analytic Query Framework

A user interacts with FastReDD by issuing a natural language analytic query Q (e.g., "What is the distribution of termination fees in software contracts from 2023?"). We execute Q over a corpus $\mathcal{D}$ of pre-segmented text chunks. While the specific chunking algorithm is orthogonal to our optimization, the chunk size creates an inherent accuracy-latency trade-off [23]; we treat this as a tunable parameter and evaluate its impact in Section 6. For consistency, we use "document" and "chunk" interchangeably to denote an atomic unit $d \in \mathcal{D}$.

We model the execution of Q as the extraction of a structured relational view from the unstructured corpus, which is then used to compute the final query. We model this transformation as a two-stage process involving a *Latent Relational View*:

$$D \xrightarrow{\text{Induce}} (S_Q, R_Q) \xrightarrow{\text{Execute}} A$$

- **Induced Schema** S_Q: This is the minimal set of relational schemas $\{s_1, s_2, \ldots, s_n\}$ discovered to be relevant to Q. Crucially, S_Q is **not** given a priori; it must be discovered from the interplay between the query and the document content.
- **Latent Database** R_Q: This is the collection of extracted relations $\{r_1, r_2, \ldots, r_n\}$ corresponding to the induced schemas. Each r_i contains tuples extracted from the corpus $\mathcal{D}$ for schema s_i.

2.2 The Oracle Baseline and Execution Bottleneck

We first model an ideal baseline using an Oracle, M_{oracle}. The M_{oracle} is a high-fidelity reasoning engine—such as a frontier LLM, a model committee, or a human expert—capable of answering complex semantic questions with gold-standard accuracy [8]. The baseline uses this Oracle for both schema induction and latent database extraction. While this guarantees a flawless extraction, executing M_{oracle} across the entire corpus incurs prohibitive latency and cost.

Prior work, such as ReDD [8], instantiates this approach by exhaustively applying an LLM committee across the corpus to materialize the latent relational view. It demonstrates the theoretical viability of an Oracle-driven approach, achieving near-perfect schema completeness, extraction and query accuracy. However, it also highlights a fundamental cost bottleneck: invoking M_{oracle} for every chunk $d \in \mathcal{D}$ forces the execution cost to scale linearly, making it intractable for large datasets.

To bridge the gap between this idealized baseline and practical resource limits, FastReDD frames execution as an error-bounded optimization problem. We introduce user-defined thresholds $\delta, T_R \in (0, 1)$ to bound the acceptable deviation from the Oracle baseline's output.

Problem 1 (Error-Bounded Cost Minimization). *Given a document corpus $\mathcal{D}$, an analytic query Q, and $\delta, T_R \in (0, 1)$, let (S_E, R_E) and (S_{oracle}, R_{oracle}) denote the schemas and tuple sets extracted by an execution plan E and the exhaustive Oracle baseline, respectively. Let $N_{oracle}(E)$ denote the total number of times the Oracle is invoked by plan E.*

The objective is to identify an execution plan E that minimizes the expected execution cost, $\mathbb{E}[N_{oracle}(E)]$, subject to bounds on schema incompleteness (δ) and extraction target recall (T_R):

$$\mathbb{P}(S_{oracle} \not\subseteq S_E) \leq \delta, \qquad \mathbb{E}\left[\frac{|R_E \cap R_{oracle}|}{|R_{oracle}|}\right] \geq T_R$$

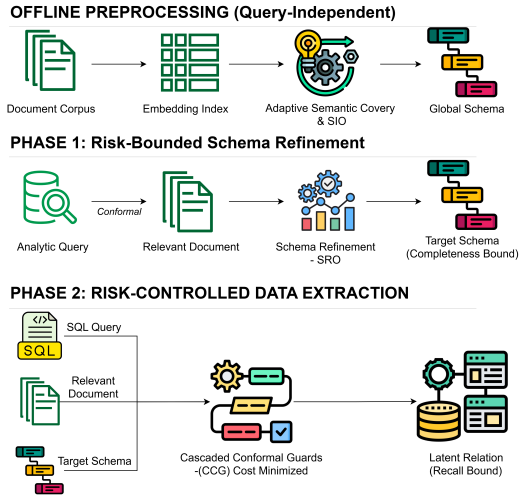


Figure 1: The FastReDD Decoupled Architecture. Offline preprocessing builds a global schema (S_{global}). Phase 1 derives a target schema (S_Q) for query Q with bounded error. Phase 2 populates the relation (R_Q) via Cascaded Conformal Guards (CCG)under recall bounds.

2.3 The FastReDD Architecture

Determining a globally optimal execution plan for Problem 1 prior to execution is inherently intractable [28]. Since the relational structure of the corpus is entirely latent, a query planner cannot estimate tuple extraction costs or evaluate the recall bound without first discovering the target attributes and their semantic selectivities. The optimization space of the extraction phase is conditioned on the schema discovery phase, making joint global optimization impossible.

To circumvent the intractability of joint global optimization, FastReDD introduces a risk-aware, decoupled execution architecture, illustrated in Figure 1. Rather than attempting to solve the intractable joint objective directly, FastReDD decomposes the query processing pipeline into three distinct functional stages:

Offline Preprocessing: Embedding and Global Induction. FastReDD computes dense vector embeddings for all document chunks in $\mathcal{D}$ to establish an embedding set $\mathcal{V}$ for indexing [25], and induces a query-independent global schema, S_{global}. Crucially, this global schema captures the macro-structure of the corpus, preventing the subsequent discovery phase from being biased by or overfitting to the limited context of only the filtered documents.

Online Phase 1: Risk-Bounded Schema Refinement. Upon receiving a query Q, the system leverages the corpus embeddings $\mathcal{V}$ to retrieve a relevant document subset, $\mathcal{D}_Q \subseteq \mathcal{D}$. It then systematically refines the global schema S_{global} against $\mathcal{D}_Q$ to produce the precise execution schema, S_Q. This step dynamically guarantees the schema risk bound without requiring a prohibitive linear scan.

Online Phase 2: Risk-Controlled Tuple Extraction. Driven by S_Q and the recall tolerance T_R, FastReDD synthesizes lightweight filtering mechanisms to safely prune irrelevant documents from $\mathcal{D}_Q$ and extract the final database R_Q. This phase tries to minimize

Algorithm 1 Adaptive Semantic Covering

Require: C (candidate set), S_0 (initial schema), Op (generative operator), ϵ (instability tolerance), w (stability window), δ_{global} (discovery risk tolerance)
Ensure: S_{final}: The fully induced and stabilized schema

```
C_obs ← ∅, S ← S_0, t ← 0
while Unified Stopping Rule is not met do
    d* ← argmax_{d∈C\C_obs} ( min_{d_j∈C_obs} ||v_d − v_{d_j}||_2 )
    C_obs ← C_obs ∪ {d*}
    t ← t + 1
    S_next ← Op(d*, S)
    Compute Jaccard instability J(t) between S and S_next
    Compute Good-Turing estimate G_est(t) for unobserved attributes
    if J(t) ≤ ε for w consecutive steps and G_est(t) ≤ δ_global then
        break
    end if
    S ← S_next
end while
return S
```

expensive Oracle invocations while guaranteeing that the expected recall loss remains bounded.

3 OFFLINE PREPROCESSING: GLOBAL SCHEMA INDUCTION

Prior to querying, FastReDD processes the corpus $\mathcal{D}$ to construct an embedding set $\mathcal{V}$ and a global schema S_{global}, absorbing structural exploration costs offline to reduce online latency bottlenecks.

3.1 Embedding Generation and the SIO Operator

We first utilize an embedding model to compute dense vector representations, $v_d \in \mathcal{V}$, for every document chunk $d \in \mathcal{D}$ [25]. $\mathcal{V}$ enables efficient semantic distance measurements for both sampling and relevant filtering.

To extract structural attributes from the text, we define the **Schema Induction Operator (SIO)**. Formally, SIO is a generative (instantiated via a generative model) operator that takes two inputs: a single unstructured document chunk $d \in \mathcal{D}$ and the current schema state S (initially empty). The operator analyzes the text in d to identify novel structural attributes not present in S, and outputs an enhanced schema S'. In our work, we instantiate SIO using the Oracle (M_{oracle}) executing a specialized prompt template[1]. Because SIO relies on this computationally expensive Oracle, evaluating $S' = \text{SIO}(d, S)$ sequentially across every document in the corpus is prohibitive. Therefore, FastReDD designs a rigorous sampling algorithm, **Adaptive Semantic Covering**, to accelerate this exploration while maintaining formal statistical guarantees (detailed in Appendix A.1).

3.2 The Adaptive Semantic Covering Algorithm

To execute schema discovery efficiently, FastReDD relies on a generalized algorithm called **Adaptive Semantic Covering** (Algorithm 1). Designed to operate across both offline and online phases, this algorithm accepts a candidate document set C (e.g., the full corpus or a dynamically filtered subset), an initial schema state, and a target generative operator Op. Crucially, the algorithm's statistical guarantee is governed by three system-defined safety parameters: the discovery risk tolerance δ_{global}, the instability tolerance ϵ and the stability window w. In the context of Embedding and Global Induction, C maps to the entire corpus $\mathcal{D}$ and Op is instantiated as the Schema Induction Operator (SIO).

The primary objective of this algorithm is to minimize the invocations of operator Op. To achieve this, it addresses two fundamental challenges: ensuring the sampled documents are diverse enough to capture all structural nuances, and rigorously determining when the schema has stabilized so exploration can safely terminate.

3.2.1 Exploration Policy. To ensure diverse document sampling and minimize redundant Op invocations, we apply a Max-Min Diversity strategy over the embedding set $\mathcal{V}$ [30]. The algorithm evaluates and selects candidate documents iteratively, one at a time. At any given step t, the system maintains a current schema state S_t and seeks a new candidate document d^*_{t+1} to read. Let $C^{(t)}_{\text{obs}}$ be the set of documents read up to step t. We select this next candidate d^*_{t+1} such that it is furthest from its nearest neighbor in the already observed set $C^{(t)}_{\text{obs}}$:

$$d^*_{t+1} = \underset{d \in C \setminus C^{(t)}_{\text{obs}}}{\text{argmax}} \left(\min_{d_j \in C^{(t)}_{\text{obs}}} \|v_d - v_{d_j}\|_2 \right) \tag{1}$$

We then invoke the generative operator, evaluated as $Op(d^*_{t+1}, S_t)$, (Op is the SIO operator in Embeddings and Global Induction) which takes both the new candidate document and the current schema state as inputs to extract or refine attributes, producing the updated schema S_{t+1} for the next step.

3.2.2 Termination Condition: Stability & Completeness. The algorithm safely terminates only when the schema is both structurally stable and statistically complete.

1. Structural Stability: We measure structural stability using the Jaccard distance between consecutive schema states at step t [14]:

$$J(t) = 1 - \frac{|F(S_t) \cap F(S_{t-1})|}{|F(S_t) \cup F(S_{t-1})|} \tag{2}$$

where $F(S_t)$ is the set of unique schema attributes present in the state S_t. To ensure true stability rather than a temporary local plateau, $J(t)$ must remain below a threshold ϵ for a sustained window of w consecutive steps.

2. Statistical Completeness (Good-Turing Estimator): To bound the risk of missing rare attributes, we estimate the probability that the next document will reveal a feature we haven't seen before. At each step t, we track the frequency of every discovered attribute. Let $T_1(t)$ be the count of "singleton" attributes—those that have been observed in exactly one document out of the t samples. The Good-Turing estimate [26] for the probability of discovering a new attribute at step $t+1$ is $G_{\text{est}}(t) = \frac{T_1(t)}{t}$

Unified Stopping Rule: The Adaptive Semantic Covering algorithm terminates at step t only when both of the following conditions are met:

(1) **Stability:** $J(t) \le \epsilon$ for w consecutive steps, where ϵ is the *instability tolerance.*
(2) **Completeness:** $G_{\text{est}}(t) \le \delta_{global}$, where δ_{global} is the *discovery risk tolerance.*

By enforcing both conditions, we rigorously bound the risk of premature termination[2]:

[1]The complete prompt template is provided in Appendix B.1.

[2]The formal proof for this theoretical guarantee is provided in Appendix A.1.

Theorem 3.1 (Dual-Convergence Guarantee). *Let $\mathcal{E}_{new}$ be the event that the next sampled document d_{t+1} introduces a previously unobserved schema feature, and $\mathcal{E}_{unstable}$ be the event that the subsequent schema state S_{t+1} diverges significantly from the current state (i.e., $J(t+1) > 2\epsilon$). Given the instability tolerance ϵ, stability window w, and discovery risk tolerance δ_{global}, if the Unified Stopping Rule is met at step t, the failure probabilities are bounded by:*

$$\mathbb{P}(\mathcal{E}_{new}) \leq \delta_{global} \quad \text{and} \quad \mathbb{P}(\mathcal{E}_{unstable}) \leq \exp\left(-\frac{w\epsilon^2}{2}\right) \tag{3}$$

3.3 Global Schema Induction Result

Upon satisfying these convergence criteria over the full corpus $\mathcal{D}$, the Adaptive Semantic Covering algorithm terminates and produces the query-independent **global schema**, S_{global}. This schema serves as a comprehensive structural guide of the entire dataset.

By materializing S_{global} offline, FastReDD ensures that the system enters the online phase with a high-recall structural foundation [8]. This pre-computation prevents the subsequent phases from being biased by the limited context of the retrieved documents, effectively mitigating the risk of missing relevant but rare attributes that were already captured in the global view.

4 RISK-BOUNDED SCHEMA REFINEMENT

The objective of this online stage is to refine the broad, offline-materialized global schema S_{global} into a precise target schema S_Q containing only the attributes necessary to resolve the user's analytic intent Q. To achieve this safely (without missing relevant attributes) and efficiently, FastReDD employs a two-step approach. First, we filter the corpus to identify a highly relevant subset $\mathcal{D}_Q$ consisting of documents that contain the specific entities, relationships, or textual evidence required to resolve the query Q. We then apply the Adaptive Semantic Covering algorithm (Algorithm 1) to refine the global schema based on the retrieved evidence.

4.1 The Conformal Semantic Filter

A fundamental challenge in the pipeline is balancing efficiency with safety: filtering candidate data too aggressively risks discarding critical evidence. To solve this, we propose the **Conformal Semantic Filter (CSF)** which is formally grounded on Conformal Risk Control (CRC) [2].

The basic functionality CSF implements is to aggressively *prune* a candidate set of documents C based on *semantic similarity* to a target embedding v_{target}, while rigorously bounding the expected loss of relevant information. The filter requires only a user-defined risk tolerance $\delta \in (0, 1)$ and a hold-out calibration dataset $\mathcal{D}_{\text{cal}} = \{(v_i, Y_i)\}_{i=1}^n$, where v_i is a target embedding and $Y_i \subseteq C$ is its ideal relevant subset (i.e., the set of documents that are semantically similar or topically aligned with v_i).[3]

Internally, the filter applies a distance threshold $\lambda \in [0, 1]$ to prune the candidates:

$$C_\lambda(v_{\text{target}}, C) = \{c \in C \mid v_{\text{target}} \cdot v_c \geq 1 - \lambda\} \tag{4}$$

To quantify missing information, CSF evaluates the **recall loss**, defined as the proportion of the ideal subset Y excluded by the threshold λ:

$$L(\lambda, v, Y) = 1 - \frac{|Y \cap C_\lambda(v, C)|}{|Y|} \tag{5}$$

[3]We detail the exact methodology for constructing this calibration set and obtaining the ground-truth labels Y_i in Section 4.2.

Algorithm 2 implements the filter. The execution of CSF proceeds in three steps. First, for each candidate threshold λ, it computes the empirical recall loss over the n examples in the calibration set $\mathcal{D}_{\text{cal}}$:

$$\hat{R}_n(\lambda) = \frac{1}{n} \sum_{i=1}^{n} L(\lambda, v_i, Y_i) \tag{6}$$

Because this empirical average often underestimates the true risk due to finite-sample variability [2], the algorithm inflates $\hat{R}_n(\lambda)$ into a rigorous upper bound $U(\lambda)$:

$$U(\lambda) = \frac{n}{n+1}\hat{R}_n(\lambda) + \frac{1}{n+1} \tag{7}$$

Next, the filter selects the optimal calibrated threshold $\hat{\lambda}$ as the minimum distance that keeps this bound at or below the user's risk tolerance δ:

$$\hat{\lambda} = \min\{\lambda \mid U(\lambda) \leq \delta\} \tag{8}$$

Finally, CSF applies $\hat{\lambda}$ to v_{target} to output the safely filtered subset. By enforcing this selected $\hat{\lambda}$, the filter statistically guarantees that the expected loss of relevant evidence on future, unseen data is rigorously bounded [2].

Algorithm 2 Conformal Semantic Filter (CSF)

Require: C (candidate embeddings), v_{target} (target vector), $\mathcal{D}_{\text{cal}} = \{(v_i, Y_i)\}_{i=1}^n$ (calibration set), $\delta \in (0, 1)$ (target risk tolerance)
Ensure: C_{filtered}: The safely pruned candidate subset

```
for each candidate threshold λ ∈ [0, 1] do
    R̂_n(λ) ← (1/n) Σ_{i=1}^n (1 − |Y_i ∩ C_λ(v_i, C)| / |Y_i|)
    U(λ) ← (n/(n+1)) R̂_n(λ) + 1/(n+1)
end for
λ̂ ← min {λ | U(λ) ≤ δ}
C_filtered ← {c ∈ C | v_target · v_c ≥ 1 − λ̂}
return C_filtered
```

4.2 Risk-Bounded Document Retrieval

To efficiently transition from the vast offline corpus $\mathcal{D}$ to a concentrated, query-specific subset, FastReDD utilizes CSF for document retrieval.

Given a user query Q, we generate its vector embedding v_Q using the same model embeddings applied to the offline corpus. We compile a representative query workload to serve as our calibration dataset $\mathcal{D}_{\text{cal}} = \{(v_{Q_i}, Y_i)\}_{i=1}^n$. These calibration queries Q_i can be either sampled from past user activity or synthetically generated offline over the corpus. For each query Q_i, Y_i denotes the ground-truth set of relevant documents. To avoid the prohibitive cost of evaluating the entire corpus, we construct Y_i offline by sampling a subset of documents. An oracle model (e.g., M_{oracle}) then processes this sample to identify the exact textual evidence required to answer Q_i.[4]

Armed with this calibration dataset and a user-defined target retrieval risk tolerance δ_{ret}, we execute the retrieval phase by invoking the CSF module (Algorithm 2) over the full corpus $\mathcal{D}$:

$$\mathcal{D}_Q = \mathcal{CSF}(\mathcal{D}, v_Q, \mathcal{D}_{\text{cal}}, \delta_{\text{ret}}) \tag{9}$$

The filter automatically calibrates the optimal similarity threshold and outputs the **Query-Relevant Evidence Set** ($\mathcal{D}_Q$). This high-recall, low-volume subset mathematically guarantees that the

[4]We perform an ablation study on the size of this calibration set n in Section 6 to evaluate its impact on performance and the tightness of our bounds.

expected missing textual evidence is bounded to at most δ_{ret} [2], ready to be passed directly into the schema refinement phase.

4.3 Adaptive Schema Refinement

With the query-relevant evidence set $\mathcal{D}_Q$, the next step is to distill the broad, offline-materialized global schema S_{global} into a precise target schema S_Q.

To do so, we define the **Schema Refinement Operator (SRO)**. Formally, SRO is a generative, reductive operator conditioned on the user's analytic intent Q. It takes two inputs: a relevant document chunk $d \in \mathcal{D}_Q$ and the current schema state S. The operator evaluates the utility of each attribute in S against the text in d to resolve the query Q, systematically pruning irrelevant attributes and refining the descriptions of the remaining ones to output a minimized schema S'. In our work, we instantiate SRO using an LLM executing a specialized, query-aware prompt template.[5]

Exhaustively running SRO sequentially across every document in $\mathcal{D}_Q$ remains computationally prohibitive; FastReDD reapplies the **Adaptive Semantic Covering** algorithm (Algorithm 1) to accelerate this refinement while maintaining formal statistical guarantees [8].

To execute this online refinement, we instantiate the Adaptive Semantic Covering algorithm over the retrieved evidence $\mathcal{D}_Q$. We initialize the *initial schema* S_0 (in Algorithm 1) using S_{global} and use as the generative operator Op the SRO. To ensure robust termination, this phase is governed by the *online discovery risk tolerance* δ_{on}. The algorithm iteratively applies the query-conditioned SRO over diverse documents from $\mathcal{D}_Q$ to systematically refine the schema attributes. The process terminates to output the finalized, minimal target schema S_Q. This query-specific schema then dictates the exact structural blueprint required for the subsequent tuple extraction phase.

5 RISK-CONTROLLED TUPLE EXTRACTION

Following the online refinement phase, FastReDD possesses a minimal, query-specific target schema S_Q and the safely bounded Query-Relevant Evidence Set $\mathcal{D}_Q$. To bridge the semantic gap between the user's natural language intent and our relational execution engine, we employ an off-the-shelf Text-to-SQL model [9, 15] to translate the user's intent into an SQL query, Q_{sql}. We assume Q_{sql} is a structurally valid representation of the user's intent.[6] Evaluating or optimizing the Text-to-SQL generation is orthogonal to our work on risk-bounded data materialization.

Once Q_{sql} is generated, FastReDD proceeds by identifying all tables referenced within the query. Rather than attempting a monolithic extraction of the entire schema simultaneously, we adopt a *table-at-a-time* extraction strategy [28]. We populate each identified table individually with structured data extracted from the unstructured documents in $\mathcal{D}_Q$. After all necessary tables are materialized, we evaluate Q_{sql} over this newly constructed relational database. Crucially, this table-at-a-time materialization approach unlocks significant execution optimization opportunities.

[5]The complete SRO prompt template is provided in Appendix B.2.

[6]Because the Schema Refinement phase provides a minimal, query-specific schema S_Q, the Text-to-SQL module operates over a highly constrained search space. This alleviates common schema-linking ambiguities and bounds the translation task [29, 39].

To capitalize on these optimizations, FastReDD introduces a **Cascaded Conformal Guards (CCG)** pipeline. Instead of passing the entirety of $\mathcal{D}_Q$ into the extraction model for every table, this pipeline leverages the logical blueprint of Q_{sql} to safely route and filter documents on a per-table basis before the table-at-a-time extraction occurs.

5.1 Semantic Table Assignment

Since tuple extraction operates at the document level and unstructured text lacks explicit relational boundaries, the mapping between documents and relational tuples is inherently complex. Primarily, it introduces a one-to-many paradigm where a single document may yield an arbitrary number of records across multiple tables. It is also possible to encounter a many-to-one scenario, where duplicated information across several documents corresponds to the exact same tuple. While FastReDD naturally tolerates these many-to-one redundancies without disruption—since all duplicates behave identically within the system—mitigating the one-to-many extraction complexity remains critical. To achieve this and enable targeted optimization, FastReDD performs an initial mapping step called **Table Assignment**. This assignment serves as a structural prerequisite, allowing us to apply independent, table-specific filters in the subsequent execution phase.

We execute this assignment by reapplying the Semantic Filter (CSF) for each table $T \in S_Q$. By embedding the schema definition of T into a vector v_T, we prune the broad evidence set $\mathcal{D}_Q$ into a table-specific subset $\mathcal{D}_T = \mathcal{CSF}(\mathcal{D}_Q, v_T, \mathcal{D}_{\text{cal}}, \delta_{\text{assign}})$(via Algorithm 2)

To parameterize this filter, we efficiently reuse the calibration dataset $\mathcal{D}_{\text{cal}}$ constructed during the calibration procedure detailed in Section 4.2. This statistically calibrated step ensures that the probability of missing a document containing valid tuples for table T is bounded by δ_{assign} [2]. Crucially, this mapping allows the system to "push down" SQL predicates from the query's WHERE clause for each table directly to the relevant document subsets, drastically reducing the volume of text subjected to the expensive generative extraction process.

5.2 Approximate Predicate Pushdown via Cascaded Conformal Guards (CCG)

Having produced table assignments organizing the corpus into table-specific subsets $\mathcal{D}_T$ for each table $T \in S_Q$, we process the extraction for each table sequentially. However, exhaustively prompting the underlying LLM to extract tuples from every document in $\mathcal{D}_T$ remains prohibitively expensive. To optimize execution, FastReDD adapts the classical database optimization of *predicate pushdown* [28] to the unstructured domain through a pipeline we call **Cascaded Conformal Guards (CCG)**. For complex queries spanning multiple tables, we cannot evaluate the entire SQL query filter conditions while extracting a single table T, because some conditions may reference attributes from other tables T' that have not yet been materialized. Thus, we apply a safe relaxation: when filtering documents for table T, we treat any predicate that depends on attributes outside of T (such as cross-table join conditions) as evaluating to True. This isolates $\mathcal{P}_T = \{p_1^T, \ldots, p_k^T\}$, which is the set of atomic SQL predicates parameterized by table T. We then use this predicate set $\mathcal{P}_T$ to filter documents prior to the tuples extraction phase.

Unstructured text lacks materialized attributes; therefore, evaluating directly whether a latent tuple satisfies a predicate $p_i^T(t)$ is difficult without first extracting the tuple. To bypass this bottleneck, CCG synthesizes lightweight surrogate models, or **LM Guards**, to approximate the presence of these predicates directly from the text. To balance computational efficiency with semantic expressiveness, we instantiate these guards using an LM-based text classification model (specifically, the `gliclass-instruct-large-v1.0` model) [33].

Formally, for each atomic predicate $p_i^T \in \mathcal{P}_T$, we define a guard function $g_i^T : \mathcal{D}_T \to \{0, 1\}$. For a given document $d \in \mathcal{D}_T$ and an atomic predicate p_i^T, the guard first produces a continuous probability score $\hat{P}_i^T(d) \in [0, 1]$ estimating whether *any* latent tuple within d satisfies p_i^T (we detail how to calibrate the discrete boolean threshold for this probability to produce the final output $g_i^T(d)$ in Section 5.2.2). Since executing these guards is orders of magnitude faster than running an LLM [33], this proxy evaluation yields substantial latency reductions.

5.2.1 Document-Level Logical Relaxation. Ultimately, the atomic guards must be composed to evaluate the full boolean WHERE clause. While the overall query defines a global condition, our multi-table relaxation allows us to independently evaluate the table-specific condition Φ_T for each table T. Since a document may contain multiple latent tuples (one-to-many document to tuple mapping), evaluating a complex condition like $\Phi_T \equiv p_1^T \wedge p_2^T$ accurately requires extracting the tuples to verify that a *single* tuple satisfies both conditions simultaneously.

To evaluate Φ_T purely at the document level without extraction, FastReDD introduces a **Document-Level Logical Relaxation**. Let T_d represent the latent (unextracted) tuples in document d. The exact tuple-level evaluation for this two-predicate conjunction requires the maximum over the joint condition: $\max_{t \in T_d} \left(p_1^T(t) \wedge p_2^T(t)\right)$

Instead of computing this exact value, our relaxation uses the discrete guards $g_1^T(d)$ and $g_2^T(d)$ to evaluate the predicates independently at the document level and compose the results. Each guard essentially approximates the document-level maximum for its respective predicate (i.e., $g_i^T(d) \approx \max_{t \in T_d} p_i^T(t)$). The relaxation checks if the document contains *some* tuple satisfying p_1^T and *some* (potentially different) tuple satisfying p_2^T:

$$g_1^T(d) \wedge g_2^T(d) \approx \left(\max_{t \in T_d} p_1^T(t)\right) \wedge \left(\max_{t' \in T_d} p_2^T(t')\right) \tag{10}$$

Formally, we define this relaxed document-level evaluation as $\hat{\Phi}_T(d)$. By substituting the exact tuple evaluations with our document-level guards, this extends to any complex table condition, evaluated as $\hat{\Phi}_T(d) = \Phi_T\left(g_1^T(d), \ldots, g_k^T(d)\right)$. Since the maximum of a conjunction over a set is mathematically bounded by the conjunction of their individual maxima, our relaxed document-level evaluation is guaranteed to serve as a upper bound to the tuple-level condition. While this relaxation may introduce False Positives (e.g., passing a document where one tuple satisfies p_1^T and a completely distinct tuple satisfies p_2^T), these are safely caught and discarded during the final generative extraction phase. Most importantly, this structural bounding mathematically guarantees that **no additional False Negatives are introduced** by the boolean composition.

5.2.2 Guard Threshold Calibration. To bridge the continuous probability output of the guards with our relaxed document-level evaluation $\hat{\Phi}_T(d)$, FastReDD needs to establish a decision threshold τ_i for each atomic predicate p_i^T. Rather than relying on arbitrary or static heuristics, we leverage Conformal prediction [31, 38] to rigorously calibrate these thresholds.

To achieve this, we construct a dedicated predicate calibration set $\mathcal{D}_{\text{cal}}^{\text{guard}}$ by prompting a front-tier LLM oracle (M_{oracle}) to perform exact tuple extraction on a small random sample of documents.[7] This establishes the ground-truth boolean labels indicating whether the document actually contains any tuple satisfying p_i^T. Using this labeled set, the conformal prediction framework [2] dynamically computes the optimal threshold τ_i.[8] It mathematically guarantees that τ_i is set at the highest possible value where the expected rate of False Negatives (i.e., erroneously discarding a document containing a valid tuple) remains bounded by the dynamically allocated risk tolerance $\delta_{p_i^T}$. During runtime execution, if the guard's estimated probability for document d exceeds τ_i, the system assigns $g_i^T(d) = 1$; otherwise, $g_i^T(d) = 0$.

During runtime execution, FastReDD approximates the relaxed upper bound by directly composing these discrete outputs. The system evaluates the composed boolean formula:

$$\hat{\Phi}_T(d) = \Phi_T\left(g_1^T(d), \ldots, g_k^T(d)\right) \tag{11}$$

Documents that fail this composed condition (i.e., $\hat{\Phi}_T(d) = 0$) are safely pruned, preserving extraction recall while drastically reducing the workload passed to the generative LLM [2]. The complete end-to-end execution of this filtering and materialization process is formalized in Algorithm 3.

5.3 The Tuple Extraction Operator (TEO)

Once the corpus is aggressively pruned by the CCG, FastReDD invokes the final materialization step on the minimized document subsets.

We formalize this phase as the **Tuple Extraction Operator (TEO)**. For a document d that successfully passes the composed guard condition (i.e., $\hat{\Phi}_T(d) = 1$) and a target table schema T, TEO explicitly invokes the generative LLM (e.g., the front-tier oracle M_{oracle}). By prompting M_{oracle} with the unstructured text of d alongside the schema constraints of T[9], TEO acts as a generative mapping from unstructured text to a finite set of materialized, structured records: $\text{TEO}(d, T) \to \{t_1, t_2, \ldots, t_m\}$, where each extracted tuple t_i matches the structural definition of T.

Crucially, because the guards $(g_1^T, \ldots, g_k^T)$ serve as an approximate document-level logical relaxation, FastReDD need perform a deterministic verification step before populating the final materialized table T. Once TEO extracts the latent tuples $\{t_1, \ldots, t_m\}$ from a given document d, the system evaluates each tuple t_i exactly against the full boolean WHERE clause Φ_T and its constituent atomic predicates $\{p_1^T, \ldots, p_k^T\}$. Only the specific tuples that satisfy the condition (i.e., $\Phi_T(t_i) = \text{True}$) are retained and inserted into T. Furthermore, this deterministic, tuple-level verification provides

[7] The complete prompt template used for ground-truth extraction is provided in Appendix B.4.
[8] We detail the specific conformal prediction techniques adopted from the literature in Section 6.
[9] The exact TEO prompt is provided in Appendix B.3.

Algorithm 3 Cascaded Conformal Guards (CCG) Pipeline

Require: Document subset $\mathcal{D}_T$, Boolean query formula Φ_T over atomic predicates $\{p_1^T, \ldots, p_k^T\}$, Target schema T
Ensure: Materialized tuples $\mathcal{T}$

```
T ← ∅
Calibrate guards {g_1^T, ..., g_k^T} and thresholds {τ_1, ..., τ_k} using conformal prediction [31, 38]
for each document d ∈ D_T do
    Compute binary guard outputs g_i^T(d) ∈ {0, 1} using thresholds τ_i
    Φ̂_T(d) ← Φ_T(g_1^T(d), ..., g_k^T(d)) {Relaxed boolean composition}
    if Φ̂_T(d) == 1 then
        T_d ← TEO(d, T) {Generative extraction via LLM}
        T ← T ∪ {t ∈ T_d | t exactly satisfies Φ_T} {Deterministic verification}
    end if
end for
return T
```

execution-time ground-truth labels for d, which are then used to drive a continuous online feedback loop that dynamically optimizes the CCG pipeline and recalibrates the guard thresholds τ_i, as described in Section 5.5.

5.4 Join-Aware Optimization: Dynamic Dictionary Guards

While the CCG pipeline efficiently filters documents for a single table, queries often require joining multiple tables (e.g., $T_{\text{parent}} \bowtie T_{\text{child}}$). We confine our presentation to the processing of a single primary foreign key join for brevity; the main ideas can be extended to multiple joins and join types.

Processing the text for each table separately is highly inefficient. It wastes expensive LLM compute extracting child records that will ultimately be thrown away because they do not match any parent record in the join [28].

To prevent this, FastReDD treats the join condition as an extra filter. We adapt a classic database optimization (*Sideways Information Passing* (SIP)) [28] for unstructured text. We use the valid join keys already found in the parent table to immediately filter the documents we search for the child table.

Let K be the join key shared between the parent table T_{parent} and the child table T_{child}. We organize the CCG pipeline to filter documents using a four-step process:

CCG with Dynamic Join Pushdown

(1) **Parent Extraction:** First, run the standard CCG pipeline and TEO for T_{parent}. This gives us a fully populated parent table instance $\mathcal{T}_{\text{parent}}$. From this table, we collect the set of valid join keys: $V_K = \pi_K(\mathcal{T}_{\text{parent}})$ where π is the projection operation.

(2) **Create Dictionary Guard:** Before starting the CCG pipeline for the child table T_{child}, we build a simple **Dictionary Guard** ($g_{\text{dict}}^{T_{\text{child}}}$) using those valid keys (V_K):

$$g_{\text{dict}}^{T_{\text{child}}}(d) = \mathbb{1}\left[\exists k \in V_K : \text{contains}(d, k)\right] \tag{12}$$

In other word, this is a semantic filter that checks if the document contains *any* valid key from the parent table.

(3) **Combine Filters:** We add this dictionary guard to the child table's existing filters as a requirement (an AND condition):

$$\hat{\Phi}'_{T_{\text{child}}}(d) = g_{\text{dict}}^{T_{\text{child}}}(d) \wedge \hat{\Phi}_{T_{\text{child}}}(d) \tag{13}$$

(4) **Child Extraction:** Finally, run the updated CCG pipeline for T_{child}. Any document that lacks a valid parent key automatically fails the filter (evaluates to 0) and is safely discarded before the expensive extraction step.

By adding g_{dict}^T into the CCG pipeline, FastReDD quickly drops "orphaned" documents. This ensures that the generative extraction operator is never applied on child text that cannot possibly survive the final relational join.

5.5 Runtime Adaptation & Continuous Calibration

FastReDD does not treat the approximate predicate pushdown pipeline as a static artifact. Instead, we implement an online feedback loop that utilizes the deterministic verification results from the TEO to continuously optimize both the execution order and the safety thresholds [44].

5.5.1 Dynamic Execution Planning (DEP). To minimize the total computational cost of evaluating the composed boolean formula Φ_T, we model the guard ordering as a dynamic scheduling problem. Because logical conjunctions ($\wedge$) can short-circuit the moment any atomic guard evaluates to $g_i^T(d) = 0$, the order in which we invoke the guards heavily dictates the overall latency [28].

We define the efficiency $E(g_i^T)$ of an atomic guard as the ratio of its empirical rejection rate to its computational execution cost:$E(g_i^T) = \frac{1-\text{pass_rate}(g_i^T)}{\text{cost}(g_i^T)}$

Initially, guards are sorted based on their performance on the warm-up calibration set. During execution, the system maintains running counters for invocations and confident rejections ($g_i^T(d) = 0$). After every batch of B documents, we recalculate $E(g_i^T)$ and re-sort the evaluation cascade. This ensures that if a guard proves highly effective at pruning the live data stream, it is automatically promoted to the front of the evaluation chain to maximize early short-circuiting [28].

5.5.2 Adaptive Conformal Calibration (ACC). Relying entirely on a static calibration set $\mathcal{D}_{\text{cal}}^{\text{guard}}$ risks violating the target extraction recall if the runtime text distribution shifts away from the initial sample. To mitigate this, FastReDD employs Online Conformal Prediction [35, 44].

During the execution phase, TEO acts as a definitive verifier for all documents that pass the upstream guards. This verification generates a continuous stream of new ground-truth labels used to dynamically update the calibration set:

- **Verified Positives:** Documents where $g_i^T(d) = 1$ and the TEO successfully extracted a tuple satisfying the predicate p_i^T.
- **Hard Negatives:** Documents where $g_i^T(d) = 1$, but the TEO extraction ultimately failed the predicate verification.

These verified samples are continuously ingested into $\mathcal{D}_{\text{cal}}^{\text{guard}}$. Periodically, we recompute non-conformity scores and update the atomic conformal thresholds τ_i. This allows to dynamically adjust decision boundaries, mathematically preserving the recall guarantee throughout the entire table population process [5].

5.6 Joint Predicate Optimization (JPO)

As illustrated in the FastReDD Decoupled Execution Architecture (Figure 1), we processes queries through a multi-stage pipeline.

In the first parts of the pipeline we extract the global schema (setting a risk tolerance δ_{global} as per Section 3.2) and the target schema given the query, setting a risk tolerance δ_{on} (as per Section 4.3). According to Problem 1 we require $\delta_{global} + \delta_{on} = \delta$.

For the next phase, of extracting the actual tuples, we have discussed calibrating the Conformal Semantic Filter (CSF) for retrieval and the Cascaded Conformal Guards (CCG) for extraction by applying individual risk tolerances, such as δ_{ret}, δ_{assign}, and δ_p, to each operator in isolation. However, because the cumulative recall of the pipeline depends on the interaction between these stages, there is a significant opportunity to optimize the system by performing a joint calibration.

Within this pipeline, stages such as Risk-Bounded Document Retrieval (using CSF), Semantic Table Assignment, and the CCG pipeline (composed of multiple atomic guards g_i) can be tuned to trade off computational cost against false negatives. This makes the allocation of risk parameters—collectively referred to here as stage-wise α values—under a global recall target T_R non-trivial.

For example, consider a set of conjunctive predicates evaluated by the CCG pipeline (Section 5.2) with per-predicate recall-loss tolerances $(\alpha_1, \alpha_2, \alpha_3)$. To meet a global reliability target of $T_R = 0.95$, a simple "uniform" allocation would enforce $\alpha_1 = \alpha_2 = \alpha_3 \approx 1 - \sqrt[3]{0.95}$. This is often inefficient because functional stages differ in semantic selectivity, calibration sharpness, and runtime cost [28]. Spending the same recall-loss allowance everywhere can waste budget on **low-effect stages**—those with weak pruning power or those not on the critical path—while over-constraining the stages that drive the most significant reductions in **Oracle** (M_{oracle}) invocations.

Motivated by this, we develop a joint α-allocation procedure that selects per-stage α values to minimize the total expected execution cost, $\mathbb{E}[N_{oracle}(E)]$, subject to the global target T_R.

5.6.1 Optimization Strategy. At a high level, we cast joint α allocation as a greedy discrete descent on cost under a recall constraint [20]. Starting from a strict, feasible configuration, we iteratively take small relaxations (moving one stage's α to the next grid value) that yield the largest cost decrease per unit increase in recall loss.

This joint allocation is tuned on a sampled subset of document chunks, denoted as D_{cache}, which is processed before the full execution of **Phase 1** or **Phase 2**. To ensure the tuning process is computationally efficient, we construct a profiling cache (Cache) on D_{cache} (Section 5.6.2) that allows us to reuse results across candidate configurations. We use θ to denote a candidate pipeline configuration, encompassing:

- The stage-wise risk tolerances (the α vector).
- The guard evaluation order Λ (Section 5.5.1).
- Any stage-specific thresholds τ_i derived from the Conformal Risk Control calibration [2].

Using this Cache, the system replays the pipeline to obtain the survivor set (documents reaching the Tuple Extraction Operator) and stage-wise invocation counts, ultimately estimating the expected per-document cost $\widehat{C}(\theta)$. The detailed steps for this procedure are discussed in Section 5.6.4.

5.6.2 Profiling Cache. To prepare for the joint α-allocation procedure (Section 5.6.4), we first construct a lightweight profiling cache from D_{cache}. We invoke the expensive **Oracle** (M_{oracle}) on D_{cache} a single time to obtain the ground-truth labels required for tuning. The resulting Cache stores, for each $d \in D_{cache}$, the scores and statistics necessary to replay each stage of the pipeline. This includes surrogate LM Guard scores (g_i), and if applicable, Conformal Semantic Filter (CSF) scores and join-related statistics, alongside their corresponding oracle-derived labels (y_i).

5.6.3 Fast Cost Computation. Given a candidate pipeline configuration θ, we estimate its expected per-document cost without re-invoking the **Oracle** by replaying the multi-stage pipeline on D_{cache} using the stored Cache. The replay process applies the Conformal Semantic Filter for retrieval, scans the Cascaded Conformal Guards in the current evaluation order Λ, and applies join-aware optimizations, terminating as soon as any stage rejects the document.

Documents that pass all enabled filtering and guard stages form the survivor set $X_{oracle} \subseteq D_{cache}$, representing the documents that would ultimately reach the Tuple Extraction Operator (TEO) for expensive processing. Let inv_{ret}, $inv[i]$, and inv_{join} denote the resulting number of invocations for the retrieval filter, the i-th atomic guard, and join-aware guards over D_{cache} under configuration θ. We then compute the estimated cost:

$$\widehat{C}(\theta) = \frac{1}{|D_{cache}|}\Big(\text{cost(CF)} \cdot \text{inv}_{\text{cf}} + \sum_{i=1}^{k} \text{cost}(g_i) \cdot \text{inv}[i] + \text{cost(Join)} \cdot \text{inv}_{\text{join}} + \text{cost}(E_{oracle}) \cdot |X_{oracle}|\Big). \tag{14}$$

This estimate serves as the primary optimization objective for joint tuning. For brevity, we denote this fast cost computation on D_{cache} using the profiling cache by $\text{CostEst}(\theta)$, which returns $\widehat{C}(\theta)$.

5.6.4 Greedy Joint Alpha Allocation. We present a greedy procedure for jointly allocating stage-wise α values—representing the risk tolerances for retrieval, assignment, and extraction—to minimize the estimated per-document cost $\widehat{C}(\theta)$. The full pseudocode of this allocation procedure is provided in Appendix C. We optimize a vector of α values: $\boldsymbol{\alpha} = (\alpha_{ret}, \alpha_1, \ldots, \alpha_k, \alpha_{join})$, where α_{ret} and α_{join} are included only if the Conformal Semantic Filter or join-aware optimizations are tunable in the specific query pipeline.

We allocate these values under a multiplicative constraint that distributes the global recall target T_R across all functional stages:

$$(1 - \alpha_{ret}) \cdot \prod_{i=1}^{k} (1 - \alpha_i) \cdot (1 - \alpha_{join}) \geq T_R \tag{15}$$

By letting $b(x) = -\log(1 - x)$ and $B = -\log(T_R)$, we can transform this into an equivalent additive form for easier optimization:

$$b(\alpha_{ret}) + \sum_{i=1}^{k} b(\alpha_i) + b(\alpha_{join}) \leq B \tag{16}$$

where missing or non-tunable stages simply drop out of the summation.

The allocation procedure takes the target T_R, evaluation order Λ, profiling cache D_{cache}, and per-stage grids as input to return the optimized $\boldsymbol{\alpha}$ allocation. The process begins with the strictest feasible allocation by setting each stage's α to its smallest grid value. While the current configuration satisfies the additive budget B, the system computes the necessary conformal thresholds and evaluates the baseline cost on D_{cache} using the fast CostEst estimator.

The algorithm then enumerates each stage and proposes the next larger α value from the grid, skipping any proposals that would violate the global risk budget. For every feasible proposal, it forms

a candidate configuration θ', recomputes the estimated cost $\widehat{C}'$, and calculates the marginal changes in cost (ΔC) and risk (Δb). If a relaxation reduces the overall cost, the proposal is scored by its efficiency: $-\Delta C/\Delta b$. The algorithm iteratively applies the most efficient relaxation until no further cost-improving steps can be taken or the risk budget is exhausted, at which point it returns the final risk allocation.

6 EXPERIMENTAL EVALUATION

In our evaluation, we benchmark and ablate the parameters of the proposed `FastReDD` system.

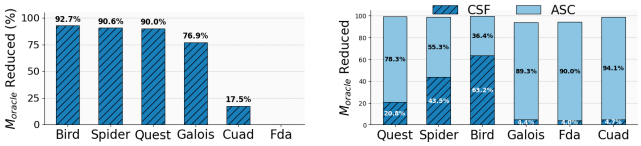
Our primary objective is to quantify the efficiency gains—measured exclusively by the reduction in Oracle (M_{oracle}) calls, which represent the computationally expensive invocations of those core generative operators (e.g., SIO, SRO, and TEO)—while maintaining a consistently controlled error rate.

6.1 Experimental Setup

6.1.1 Benchmark Datasets. To thoroughly evaluate FastReDD's robustness, extraction accuracy, and runtime efficiency, the system is benchmarked across several highly diverse datasets spanning multiple domains. These include collections derived from structured Text-to-SQL benchmarks (Spider, BIRD) alongside complex real-world unstructured corpora such as legal contracts (CUAD, QUEST-LCR), regulatory documents (FDA), Wikipedia articles (QUEST-Wiki), and finance or sports reports (Galois). Consistent with the focus on risk-bounded materialization rather than intent translation, extraction performance is isolated across all datasets by providing the system with the gold-standard SQL queries [9]. Full details regarding the dataset compilation, structural complexity, and the specific prompts used for data synthesis are provided in Appendix D.

- **Spider & BIRD:** Derived from the widely used Spider and BIRD Text-to-SQL benchmarks [8, 16, 45], these mixed-domain datasets transform structured data into unstructured document collections. Because they originate from relational databases, the exact SQL execution results are known, providing a reliable ground truth. Spider comprises 2,673 documents evaluating 10–69 attributes, while BIRD contains 1,500 documents with 21–82 attributes.
- **QUEST:** Adapted from real-world unstructured data analysis literature [1, 19], this suite includes **LCR** (legal domain, 600 documents, 10 attributes); **WikiText** (Wikipedia pages with natural hierarchical structures, 3,900 documents, 33 attributes); and **SWDE** (structurally dense web pages, 100 documents, 3–13 attributes).
- **CUAD & FDA:** Selected to represent extremely long, real-world texts [12, 43]. CUAD features legal contracts averaging roughly 10,000 words (500 documents, 40 attributes), while FDA focuses on regulatory documents (100 documents, 16 attributes).
- **Galois:** Sourced from the Fortune and Premier League datasets [28], this collection targets the financial and sports domains. It consists of 542 medium-length articles requiring the precise extraction of 7–29 attributes.

Consistent with the focus on risk-bounded materialization rather than intent translation, extraction performance is isolated across all datasets by providing the system with the gold-standard SQL queries [9].



(a) Global Schema Induction **(b) Schema Refinement**

Figure 2: Percentage of exhaustive M_{oracle} calls reduced on schema discovery.

6.1.2 System Implementation. All local model inference and pipeline evaluations are conducted on a dedicated server node equipped with one NVIDIA RTX 5090 GPU, 512GB of RAM, and an AMD EPYC 128-core processor. The `FastReDD` framework is implemented in Python 3.10 [8]. For our core models, we instantiate the exhaustive Oracle (M_{oracle}) and the corresponding operators (SIO, SRO, TEO) using the Gemini-3-Pro API [11]. The dense vector representations required by the Conformal Semantic Filter (CSF) are generated using the `text-embedding-3-large` model [25]. Finally, the surrogate LM Guards utilized in the Cascaded Conformal Guards (CCG) pipeline are instantiated using the `gliclass-instruct-large` [33] model. Unless otherwise specified, default parameters are set as follows: schema instability convergence threshold $\epsilon = 0.01$ with a window size of 5 (offline) and 1 (online refinement), and a 0.95 target confidence level for all conformal filters.

6.1.3 Evaluation Metrics. To evaluate the trade-off between extraction cost and fidelity, we rely on the following metrics:

- **Efficiency (N_{calls} & ρ):** The absolute number of generative LLM invocations (N_{calls}), and the relative reduction rate $\rho = 1 - (N_{\text{FR}}/N_{\text{BF}})$, where N_{FR} and N_{BF} represent total invocations executed by FastReDD and the Brute Force baseline, respectively.
- **Fidelity (P_E, R_E & $\hat{\alpha}$):** Let $E \in \{\mathcal{S}, \mathcal{T}\}$ denote schema attributes or tuple records. We measure precision and recall. To verify the tuple extraction process, we measure the empirical recall loss against the ground truth $\hat{\alpha} = 1 - \text{recall}$.

6.2 Evaluating Schema Discovery Efficiency

To comprehensively evaluate the computational efficiency of the FastReDD framework, we first establish a practical setting focused on the schema discovery pipeline. Specifically, we bound the target discovery error tolerance (δ) for schema discovery at 0.05 (with 0.04 for δ_{global} and 0.01 for δ_{on}). Under this fixed risk threshold, we measure the reduction in exhaustive M_{oracle} invocations alongside the final extraction accuracy for the schema discovery phase.We will conclude this section by evaluating the impact of varying the target discovery error tolerance on the framework's overall performance.

6.2.1 Efficiency Under Bounded Error.

Offline Global Induction Global schema induction serves as an offline preprocessing step to construct a comprehensive structural view of the underlying corpus. By restricting the input of synthesis operators to high-confidence, refined evidence, our framework significantly accelerates this phase. Figure 2(a) reports the percentage of exhaustive M_{oracle} invocations bypassed. FastReDD achieves substantial reductions on structured and semi-structured benchmarks, bypassing 92.7%, 90.6%, and 90.0% of oracle calls on *Bird*, *Spider*, and *Quest*, respectively, while maintaining a 76.9% reduction on *Galois*. This confirms that early pruning effectively eliminates redundant

Table 1: Schema Refinement Precision (P) and Recall (R) scores at the Table and Attribute levels. `FastReDD` achieves a perfect 1.0 recall across all benchmarked datasets.

Dataset	Table P	Table R	Attr P	Attr R
Quest	0.988	1.0	0.988	1.0
Spider	0.969	1.0	0.982	1.0
BIRD	0.722	1.0	0.602	1.0
Galois	1.000	1.0	0.775	1.0
Fda	1.000	1.0	0.834	1.0
Cuad	1.000	1.0	0.75	1.0

context, minimizing reliance on the heavyweight generative model for global synthesis.

Conversely, efficiency on highly unstructured, domain-specific text relies heavily on corpus scale and structural regularity. Reductions in exhaustive calls drop to 17.5% on *Cuad* and near-zero on *Fda*. In dense legal or medical documents, ambiguous semantic boundaries necessitate deeper reasoning. Furthermore, *Fda*'s minimal scale (100 documents) lacks the structural redundancy required to establish reusable patterns, forcing FastReDD to rely on exhaustive M_{oracle} invocations across almost all available context. Crucially, however, the primary utility of this phase is realized at query time. Even when offline preprocessing yields modest speedups on complex corpora, materializing the global schema remains essential; it provides a macro-level view S_{global} that drastically accelerates and refines the online derivation of the target schema when a user query arrives.

Online Schema Refinement Following the global schema induction phase, the Schema Refinement step leverages the newly established S_{global} to perform highly aggressive, lossless pruning. Because the offline global induction already provides a comprehensive macro-level understanding of the document's structural boundaries, this subsequent refinement step is exceptionally fast and accurate. We evaluate this phase along two dimensions: the computational efficiency gained by bypassing the generative model, and the accuracy of the resulting refined schema.

Figure 2(b) details the percentage of M_{oracle} calls bypassed, segmented by our two primary pruning mechanisms: the Conformal Semantic Filter and Adaptive Semantic Covering. Across all benchmarks, `FastReDD` achieves near-total reduction, effectively eliminating over 93% of expensive oracle calls. However, the proportional contribution of these mechanisms shifts drastically depending on structural complexity. On highly complex relational databases like *BIRD*, the Conformal Semantic Filter drives the optimization, accounting for 63.2% of the reduction. Because relational datasets possess rigid constraints and dense metadata, the conformal filter utilizes S_{global} to confidently rule out irrelevant schema elements early. Conversely, on unstructured, domain-specific text (*Galois*, *Fda*, and *Cuad*), the Conformal Semantic Filter's contribution shrinks below 5%. In these ambiguous environments, Adaptive Semantic Covering dynamically takes over, identifying semantic equivalencies to prune between 89.3% and 94.1% of the search space.

Crucially, these massive efficiency gains do not compromise the integrity of the target context. As shown in Table 1, `FastReDD` achieves a perfect 1.0 recall at both the table and attribute levels across every tested dataset. This guarantees strictly lossless refinement; no relevant ground-truth schema elements are discarded during pruning. Precision remains remarkably high for the majority of datasets, with an expected moderate decrease at the attribute level for *BIRD* (0.602). Given *BIRD*'s uniquely massive schema scale, ensuring perfect recall necessitates a slightly more conservative pruning boundary.

Table 2: Sensitivity analysis of δ thresholds.N_{calls}, represents the number of oracle invocationsduring the online schema refinement phase.

1-δ		Accuracy	Efficiency
1-δ_{global}	1-δ_{online}	Avg Attr R	Avg N_{calls}
Varying global tolerance (Fixed δ_{online} = 0.01)			
0.90	0.99	0.99	34.6
0.92	0.99	0.99	24.8
0.96	**0.99**	**1.00**	**5.2**
Varying online tolerance (Fixed δ_{global} = 0.04)			
0.96	0.90	0.99	3.9
0.96	0.95	1.00	4.8
0.96	**0.99**	**1.00**	**5.2**

6.2.2 Sensitivity to Error Tolerance. While our framework employs two error thresholds, their practical impacts differ significantly (Table 2). Offline global induction requires a strict threshold (δ_{global}) to construct a high-recall structural view (S_{global}). Relaxing δ_{global} forces the system to exhaustively reconstruct missing context on the fly. Although the high online tolerance ultimately preserves final extraction recall, this recovery process severely degrades query-time efficiency, causing the average number of online oracle calls (N_{calls}) to spike dramatically as the system compensates for the weak global view.

Conversely, online schema refinement is remarkably insensitive to variations in δ_{online}. Adjusting this parameter yields negligible fluctuations in processing speed—averaging around 4 to 5 oracle calls—while maintaining near-perfect accuracy. This robustness stems from leveraging the pre-computed S_{global} over query-relevant documents, which largely eliminates the need to discover novel attributes from scratch. Because the algorithm consistently observes recurring schema elements and lacks a long tail of unpredictable "singleton" attributes, it rapidly achieves convergence. Consequently, online termination is overwhelmingly dominated by the framework's early stability condition rather than statistical bounds. Based on these findings, we recommend configuring $\delta_{global} \leq 0.04$ to ensure a robust structural foundation, and comfortably setting $\delta_{online} = 0.01$ to maximize extraction fidelity without incurring unnecessary overhead.

6.3 Risk-Controlled Tuple Extraction Evaluation

This subsection evaluates Online Phase 2 in isolation. Unless otherwise stated, we fix the global tuple target recall to $T_R = 0.95$ (target error bound $1 - T_R = 0.05$). Importantly, all tuple-side measurements include the full pipeline effects from both **CSF** (Section 4.1) and **CCG** (Section 5.2): CSF changes the table-specific candidate set before extraction, and CCG includes join-aware guards (e.g., dictionary guards) in addition to predicate guards.

6.3.1 End-to-End Efficiency at Fixed Error. Table 3 reports Oracle LLM (Gemini-3-Pro [11]) call volume N_{calls} under fixed $T_R = 0.95$, where ReDD-BF denotes the exhaustive ReDD brute-force baseline [8] that invokes the Oracle tuple extractor on every candidate

Table 3: Tuple extraction efficiency at fixed target recall $T_R = 0.95$ (target error bound $1 - T_R = 0.05$).

Dataset	ReDD-BF	FastReDD	Call Reduction (ρ)
Quest	3342	622	81.4%
Spider	1733	471	72.8%
BIRD	2435	567	76.7%
Galois	542	113	79.1%
FDA	727	119	83.6%
CUAD	500	55	88.9%

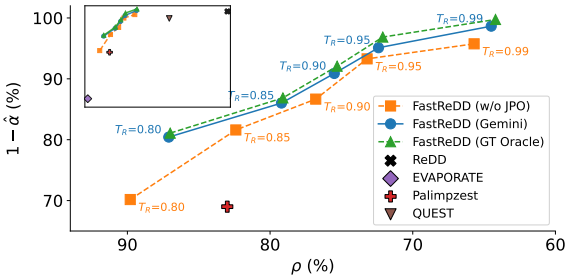


Figure 3: Impact of tuning target recall T_R on tuple extraction. We report empirical recall $1 - \hat{\alpha}$ and call reduction ρ.

document without any pruning. Across all datasets, `FastReDD` substantially reduces expensive Oracle calls while satisfying the target error bound. The reduction is consistent across data sets, ranging from 72.8% to 88.9%. In absolute terms, Oracle calls drop substantially (e.g., BIRD: 2435 → 567), confirming that risk-controlled filtering removes a large fraction of unnecessary extraction calls under the same target error bound.

6.3.2 Sensitivity to Target T_R. Figure 3 sweeps the global target recall T_R in FastReDD and reports the resulting *empirical recall* ($1 - \hat{\alpha}$) and *oracle call reduction* (ρ). Tightening T_R yields a clear cost–accuracy frontier: higher recall is achieved at the expense of pruning fewer oracle calls. The trade-off is monotonic on our main curve FastReDD (Gemini): increasing T_R from 0.80 to 0.99 raises $1-\hat{\alpha}$ from 80.43% to 98.65%, while ρ decreases from 87.1% to 64.5%. Compared to the variant without joint risk allocation (CSF+CCG w/o JPO), FastReDD consistently achieves higher recall at comparable ρ; for example, at $T_R = 0.95$ it improves recall from 93.28% to 95.11% with similar call reduction (73.2% vs. 72.4%). Figure 3 also overlays a Ground Truth (FastReDD GT Oracle) upper-bound curve and baselines for reference. The GT curve is an oracle upper bound using ground-truth tuples; its proximity to Gemini-3-Pro-based FastReDD suggests that with a modern model, FastReDD already approaches this upper bound performance. We include representative baselines[10]: **ReDD** (brute-force; highest accuracy but near-exhaustive calls [8]), **EVAPORATE**/**Palimpzest** (program generation/reuse to amortize reasoning, but with lower recall [3, 20]), and **QUEST** (optimized execution, but with room for further improvement [34]).

6.3.3 Ablation Study. Figure 4 reports the ablation results using the same tuple-side metrics (ρ, $1 - \hat{\alpha}$) under fixed $T_R = 0.95$. In this figure, each label denotes a FastReDD variant for tuple extraction: **CSF** means only CSF is enabled; **CCG** means only CCG is enabled (including the join-aware dictionary guard optimization from Section 5.4); **CSF+CCG** means both CSF and CCG are enabled

[10]Since the main panel zooms into the high-recall regime to highlight FastReDD's frontier, baselines whose operating points fall far outside this range (e.g., EVAPORATE and QUEST) are shown in the inset for completeness.

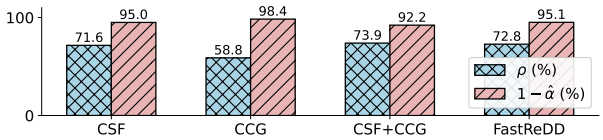


Figure 4: Ablation Study. CSF: only CSF enabled; CCG: only CCG enabled; CSF+CCG: both enabled but without JPO.

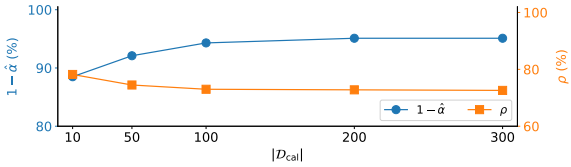


Figure 5: Tuple extraction sensitivity to calibration set size $|\mathcal{D}_{\text{cal}}|$.

but without Joint Predicate Optimization (JPO; Section 5.6); and **FastReDD** adds JPO on top of CSF+CCG. CSF-only achieves 71.6% call reduction with 95.0% recall by removing irrelevant chunks before table assignment. CCG-only achieves 58.8% call reduction with 98.4% recall via predicate-aware screening (including join-aware filtering). Combining CSF+CCG yields the highest raw pruning (73.9%) but lowers recall to 92.2%. Adding JPO in full `FastReDD` improves recall to 95.1% with only a small reduction change (72.8%), showing that joint risk allocation better balances cost and accuracy.

6.3.4 Effect of Calibration Size. Here, $\mathcal{D}_{\text{cal}}$ denotes the pooled calibration set in tuple extraction, i.e., the union of stage-specific calibration subsets used by the filtering/calibration components (CSF, CCG, as well as JPO). Therefore, $|\mathcal{D}_{\text{cal}}|$ is the total number of calibration samples across these calibration sets. Figure 5 varies $|\mathcal{D}_{\text{cal}}|$ on Spider and shows the trade-off between calibration quality and cost. Increasing calibration size improves empirical recall and stabilizes decision thresholds; after a moderate size, gains saturate while cost reduction remains stable. We observe the same diminishing-returns trend on the other datasets (QUEST, BIRD, Galois, FDA, and CUAD; not shown), and thus use a moderate calibration size for all tuple extraction experiments unless otherwise stated. The curve shows clear diminishing returns: recall improves from 88.5% (size 10) to 95.1% (size 200), while call reduction changes from 78.2% to 72.8%. Beyond this point (size 300), both metrics are nearly unchanged, indicating that a moderate calibration set is sufficient in practice.

6.3.5 Chunk-to-Table Mapping Patterns. Figure 6 compares one-to-one, one-to-many, and many-to-one document-to-tuple/table mappings. The one-to-many case is already explicitly handled by our tuple extraction design (Section 5.1), where a single document can generate multiple tuples and table assignments. In this evaluation, the many-to-one case specifically refers to corpora with update/delete-style events where multiple chunks map to the same logical tuple; we manually construct this setting to test robustness under duplicate and overwrite-like evidence. The results show that FastReDD remains stable across all three mapping regimes, indicating that table assignment and guard composition are not brittle to mapping multiplicity. Across the three mapping regimes, variation is small (recall: 95.1%, 95.0%, 94.8%; call reduction: 72.8%, 72.5%, 72.5%), showing that tuple-side behavior is robust to chunk-to-table cardinality patterns.

6.3.6 Effect of Dynamic Execution Planning (DEP). Dynamic Execution Planning (DEP; Section 6.3.6) via dynamic guard re-ordering

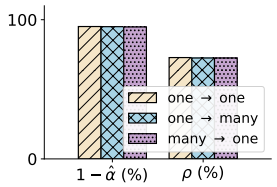


Figure 6: Tuple extraction under diff chunk-to-table mapping.

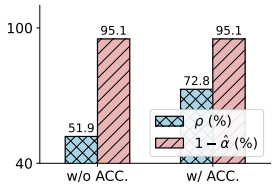


Figure 7: Tuple extraction with Adaptive Conformal Calibration.

does *not* change the tuple-level quality/cost metrics reported in Section 6.3.1: it does not affect Oracle call volume N_{calls}, call reduction rate ρ, or empirical error rate $\hat{\alpha}$. Its effect is only on guard-side inference latency. Since guard inference is much cheaper than Oracle calls, this optimization is secondary at the full pipeline level. Even so, dynamic re-ordering still yields measurable speedups for the guard stage by promoting high-rejection guards earlier in the short-circuit cascade. The gain depends primarily on predicate count in the query: for queries with 2 predicates, average guard-side latency improves by 26%; for queries with 3 or more predicates, the average improvement increases to 41%.

6.3.7 Effect of Adaptive Conformal Calibration (ACC). We further evaluate online calibration updates using execution-time verification labels. Adaptive Conformal Calibration (ACC; Section 6.3.7) improves robustness against distribution shift after warm-up: thresholds are tightened or relaxed based on newly verified positives and hard negatives, which keeps empirical error $\hat{\alpha}$ aligned with the configured bound $1 - T_R$ throughout long-running tuple scans. To stress-test this behavior, we manually construct a distribution-shift setting on ReDD-Spider[11] and compare execution with and without Adaptive Conformal Calibration (ACC; Section 6.3.7). Figure 7 shows that ACC preserves the same recall level ($1 - \hat{\alpha} = 95.1\%$) while substantially improving call reduction (from 51.9% to 72.8%). This adaptive update is particularly important for long executions, where static warm-up thresholds can become stale; continuously refreshing calibration with verified feedback maintains stable safety-quality behavior without re-running full offline calibration.

6.4 Wall Clock Time Execution Time

Finally, we report on the end-to-end wall clock time of the `FastReDD` pipeline. In this work, overall execution latency is strictly bounded by the remote API inference costs associated with the generative model, which typically require 15 to 30 seconds per call. For the tuple extraction phase, this latency is highly dependent on the complexity and selectivity of the query: queries that demand a higher volume of extracted tuples naturally require evaluating a larger subset of candidate documents, thereby linearly increasing the total API wait time.

It is important to note that the primary objective of our current implementation is to algorithmically reduce the absolute volume of Oracle calls rather than to engineer system-level concurrency. Consequently, we do not optimize API usage to saturate the theoretical Tokens Per Minute limits outlined by the provider. Instead, all Oracle invocations are executed sequentially. Under this sequential paradigm, the average runtime for the online schema refinement phase is approximately 60 seconds. Conversely, the query-dependent tuple extraction phase exhibits a much wider variance, with total runtime varying from 1 minute to 1 hour. This upper bound is typical of low-selectivity aggregation queries, which require extracting event-based tuples across hundreds of relevant documents.

[11]We induce a distribution shift by sorting the documents into two segments with markedly different predicate selectivity (i.e., different match rates).

7 RELATED WORK

LLM-Powered Unstructured Analytics Systems. Large Language Models (LLMs) have enabled a new generation of systems to transform heterogeneous documents into structured, queryable views. Representative examples include EVAPORATE [3], TWIX [18], ZENDB [19], DocETL [32], Pneuma [4], and broader query engines like UQE, ELEET, Unify, and iDataLake [10, 37, 40, 41]. Recent VLDB/ICDE optimizers further strengthen this direction, including QUEST [34], Doctopus [7], DocDB [17], and DataLab [42]. Despite this progress, most systems prioritize execution efficiency and utility, lacking an explicit, end-to-end recall-loss control mechanism for query answers over unstructured corpora.

Declarative Optimization for AI-Driven Query Processing. Recent work has pushed LLM-centric analytics toward declarative and optimizer-driven execution, including Palimpzest [20–22], logical/physical optimizations [28], and semantic operator abstractions [27]. Systems like CAESURA and TAG integrate LLM planning into query workflows [6, 36], while agentic frameworks like DSPy and LangChain optimize prompt composition [13, 24]. In contrast, our focus is on risk-bounded relational materialization over unstructured data; our objective goes beyond cost minimization to include strict adherence to a user-controlled recall-loss budget.

Text-to-SQL. Text-to-SQL research translates natural-language intents into executable SQL over existing schemas, utilizing benchmarks and methods like Spider, RAT-SQL, PICARD, and RESDSQL [15, 29, 39, 45]. Recent reliability-oriented methods also improve robustness via calibrated abstention [9]. These advances are orthogonal and composable with our setting: once query-ready schemas are available, these models serve as the front-end semantic interface. Our work instead targets the upstream problem of discovering and populating those relational structures from unstructured evidence under explicit error control.

Conformal Prediction and Risk Control. Conformal prediction provides distribution-free uncertainty control with finite-sample guarantees [31, 38]. Building on this, Conformal Risk Control and its extensions provide tools for directly controlling task-level risk, even under distribution shifts [2, 5, 35, 44]. We adapt these concepts into unstructured query processing operators, using conformal calibration to bound recall loss across retrieval, filtering, and extraction stages rather than merely calibrating isolated model confidence.

8 CONCLUSIONS

In this paper, we presented FastReDD, an error-aware query processing framework that accelerates relational analytics over unstructured text by treating frontier LLMs as a scarce resource. Our extensive evaluation demonstrates that our decoupled, risk-bounded architecture reduces frontier model invocations by up to an order of magnitude without violating user-defined extraction recall targets. In future work, we plan to extend our dynamic join-aware optimizations to support more complex multi-way joins and diverse join types across heterogeneous unstructured corpora.

REFERENCES

[1] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Benjamin Sowell, Dan Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. arXiv:2409.00847 [cs.DB] https://arxiv.org/abs/2409.00847

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal Risk Control. In *The Twelfth International Conference on Learning Representations*. https://arxiv.org/abs/2208.02814

[3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *arXiv preprint arXiv:2304.09433* (2023). https://arxiv.org/abs/2304.09433

[4] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[5] Rina Foygel Barber, Emmanuel J Candes, Aaditya Ramdas, and Ryan J Tibshirani. 2023. Conformal prediction beyond exchangeability. *The Annals of Statistics* 51, 2 (2023), 816–845.

[6] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2SQL is Not Enough: Unifying AI and Databases with TAG. arXiv:2408.14717 [cs.DB] https://arxiv.org/abs/2408.14717

[7] Chengliang Chai, Jiajun Li, Yuhao Deng, Yuanhao Zhong, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Doctopus: Budget-aware Structural Table Extraction from Unstructured Documents. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3695–3707. doi:10.14778/3749646.3749647

[8] Daren Chao, Kaiwen Chen, Naiqing Guan, and Nick Koudas. 2025. Relational Deep Dive: Error-Aware Queries Over Unstructured Data (Technical Report). https://www.cs.toronto.edu/~drchao/papers/2025_ReDD_TechReport.pdf.

[9] Kaiwen Chen, Yueting Chen, Nick Koudas, and Xiaohui Yu. 2025. Reliable Text-to-SQL with Adaptive Abstention. *Proc. ACM Manag. Data* 3, 1, Article 69 (Feb. 2025), 30 pages. doi:10.1145/3709719

[10] Hanjun Dai, Bethany Yixin Wang, Xingchen Wan, Bo Dai, Sherry Yang, Azade Nova, Pengcheng Yin, Phitchaya Mangpo Phothilimthana, Charles Sutton, and Dale Schuurmans. 2024. UQE: A Query Engine for Unstructured Databases. arXiv:2407.09522 [cs.DB] https://arxiv.org/abs/2407.09522

[11] Google. 2025. Gemini API Documentation. https://ai.google.dev/gemini-api/docs.

[12] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An expert-annotated NLP dataset for legal contract review. *arXiv preprint arXiv:2103.06268* (2021). https://arxiv.org/abs/2103.06268

[13] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714 [cs.CL] https://arxiv.org/abs/2310.03714

[14] Sven Kosub. 2019. A note on the triangle inequality for the Jaccard distance. *Pattern Recognition Letters* 120 (apr 2019), 36–38. doi:10.1016/j.patrec.2018.12.007

[15] Honghao Li, Jiajun Zhao, Tao Yu, Rui Zhang, and Dragomir Radev. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13314–13322.

[16] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.

[17] Zequn Li, Yuanhao Zhong, Chengliang Chai, Zhaoze Sun, Yuhao Deng, Ye Yuan, Guoren Wang, and Lei Cao. 2025. DocDB: A Database for Unstructured Document Analysis. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5387–5390. doi:10.14778/3750601.3750678

[18] Yiming Lin and et al. 2025. TWIX: Automatically Reconstructing Structured Data from Templatized Documents. *arXiv preprint arXiv:2501.06659* (2025). arXiv:2501.06659 [cs.CL]

[19] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G. Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. arXiv:2405.04674 [cs.DB] https://arxiv.org/abs/2405.04674

[20] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpzest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.

[21] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baille Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, and Gerardo Vitagliano. 2024. A Declarative System for Optimizing AI Workloads. arXiv:2405.14696 [cs.CL] https://arxiv.org/abs/2405.14696

[22] Chunwei Liu, Gerardo Vitagliano, Brandon Rose, Matthew Printz, David Andrew Samson, and Michael Cafarella. 2025. PalimpChat: Declarative and Interactive AI analytics. In *Companion of the 2025 International Conference on Management of Data*. 183–186.

[23] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. Web publication at informationretrieval.org..

[24] Vasilios Mavroudis. 2024. LangChain v0.3. HAL preprint. https://hal.science/hal-04817573v1

[25] OpenAI. 2024. New embedding models and API updates. https://openai.com/index/new-embedding-models-and-api-updates/.

[26] Alon Orlitsky, Narayana P. Santhanam, and Junan Zhang. 2003. Always Good Turing: Asymptotically Optimal Probability Estimation. *Science* 302, 5644 (oct 2003), 427–431. doi:10.1126/science.1088284

[27] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. arXiv:2407.11418 [cs.DB] https://arxiv.org/abs/2407.11418

[28] Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. 2025. Logical and Physical Optimizations for SQL Query Execution over Large Language Models. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[29] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 9895–9901. doi:10.18653/v1/2021.emnlp-main.779

[30] Ozan Sener and Silvio Savarese. 2018. Active Learning for Convolutional Neural Networks: A Core-Set Approach. In *International Conference on Learning Representations*. https://openreview.net/forum?id=H1aIuk-RW

[31] Glenn Shafer and Vladimir Vovk. 2008. A tutorial on conformal prediction. *Journal of Machine Learning Research* 9, 3 (2008).

[32] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3035–3048. doi:10.14778/3746405.3746426

[33] Ihor Stepanov, Mykhailo Shtopko, Dmytro Vodianytskyi, Oleksandr Lukashov, Alexander Yavorskyi, and Mykyta Yaroshenko. 2025. GLiClass: Generalist Lightweight Model for Sequence Classification Tasks. *arXiv preprint arXiv:2508.07662* (2025). https://arxiv.org/abs/2508.07662

[34] Zhaoze Sun, Chengliang Chai, Qiyan Deng, Kaisen Jin, Xinyu Guo, Han Han, Ye Yuan, Guoren Wang, and Lei Cao. 2025. QUEST: Query Optimization in Unstructured Document Analysis. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4560–4573. doi:10.14778/3749646.3749713

[35] Ryan J. Tibshirani, Rina Foygel Barber, Emmanuel J. Candès, and Aaditya Ramdas. 2019. Conformal Prediction Under Covariate Shift. In *Advances in Neural Information Processing Systems*.

[36] Matthias Urban and Carsten Binnig. 2024. Demonstrating CAESURA: Language Models as Multi-Modal Query Planners. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 472–475. doi:10.1145/3626246.3654732

[37] Matthias Urban and Carsten Binnig. 2024. Efficient Learned Query Execution over Text and Tables [Technical Report]. arXiv:2410.22522 [cs.DB] https://arxiv.org/abs/2410.22522

[38] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic learning in a random world*. Vol. 29. Springer.

[39] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 7567–7578. doi:10.18653/v1/2020.acl-main.677

[40] Jiayi Wang and Jianhua Feng. 2025. Unify: An Unstructured Data Analytics System . In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Los Alamitos, CA, USA, 4662–4674. doi:10.1109/ICDE65448.2025.00374

[41] Jiayi Wang, Guoliang Li, and Jianhua Feng. 2025. iDataLake: An LLM-Powered Analytics System on Data Lakes. *IEEE Data Eng. Bull.* 49, 1 (2025), 57–69. http://sites.computer.org/debull/A25mar/p57.pdf

[42] Luoxuan Weng, Yinghao Tang, Yingchaojie Feng, Zhuo Chang, Ruiqin Chen, Haozhe Feng, Chen Hou, Danqing Huang, Yang Li, Huaming Rao, Haonan Wang, Canshi Wei, Xiaofeng Yang, Yuhui Zhang, Yifeng Zheng, Xiuqi Huang, Minfeng Zhu, Yuxin Ma, Bin Cui, Peng Chen, and Wei Chen. 2025. DataLab: A Unified Platform for LLM-Powered Business Intelligence. In *41st IEEE International Conference on Data Engineering (ICDE)*. IEEE, 4346–4359. doi:10.1109/ICDE65448.2025.00326

[43] Eric Wu, Kevin Wu, Roxana Daneshjou, David Ouyang, Daniel E Ho, and James Zou. 2021. How medical AI devices are evaluated: limitations and recommendations from an analysis of FDA approvals. *Nature Medicine* 27, 4 (2021), 582–584.

[44] Fatih Furkan Yilmaz and Reinhard Heckel. 2023. Test-time Recalibration of Conformal Predictors Under Distribution Shift Based on Unlabeled Examples. arXiv:2210.04166 [cs.LG] https://arxiv.org/abs/2210.04166

[45] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887* (2018).

A FORMAL PROOFS AND THEORETICAL GUARANTEES

A.1 Proof of Theorem 3.1 (Dual-Convergence Guarantee)

Theorem 3.1 Let $\mathcal{E}_{new}$ be the event that the next sampled document d_{t+1} introduces a previously unobserved schema feature, and $\mathcal{E}_{unstable}$ be the event that the subsequent schema state S_{t+1} diverges significantly from the current state (i.e., $J(t+1) > 2\epsilon$). Given the instability tolerance ϵ, stability window w, and discovery risk tolerance δ_{global}, if the Unified Stopping Rule is met at step t, the failure probabilities are bounded by:

$$\mathbb{P}(\mathcal{E}_{new}) \leq \delta_{global}$$

$$\mathbb{P}(\mathcal{E}_{unstable}) \leq \exp\left(-\frac{w\epsilon^2}{2}\right)$$

Proof. **Completeness Bound:** Let $T_1(t)$ denote the count of singleton attributes observed exactly once after t document chunks. Utilizing the Good-Turing estimator, the probability mass of discovering a previously unobserved attribute in the next sample is estimated as $G_{est}(t) = \frac{T_1(t)}{t}$. The algorithm's Unified Stopping Rule mandates that exploration continues until $G_{est}(t) \leq \delta_{global}$. Therefore, the expected probability of event $\mathcal{E}_{new}$ (encountering a new attribute at step $t+1$) is directly bounded by the user-defined discovery risk tolerance:

$$\mathbb{P}(\mathcal{E}_{new}) \leq \delta_{global}$$

Stability Bound: Let $J(t)$ represent the Jaccard distance between consecutive schema states S_t and S_{t-1}. We observe $J(t)$ over a stability window of w consecutive steps. To bound the probability of a sudden structural divergence (event $\mathcal{E}_{unstable}$ where $J(t+1) > 2\epsilon$), we model the sequence of Jaccard distances as a bounded martingale difference sequence, noting that $J(t) \in [0, 1]$. By the stopping condition, the system has observed a consecutive window of length w where the empirical instability satisfies $J(t) \leq \epsilon$. Applying Hoeffding's inequality to bound the deviation of the true expected instability from this observed empirical upper bound, the probability that the underlying instability exceeds 2ϵ decays exponentially with the window size. Consequently, the probability of $\mathcal{E}_{unstable}$ is strictly bounded by:

$$\mathbb{P}(\mathcal{E}_{unstable}) \leq \exp\left(-\frac{w\epsilon^2}{2}\right)$$

By satisfying both conditions simultaneously, the Adaptive Semantic Covering algorithm ensures that the probability of premature termination is rigorously bounded. □

B GENERATIVE MODEL PROMPTS

The core generative operators in FastReDD rely on the front-tier Oracle model (M_{oracle}).

B.1 Schema Induction Operator (SIO) Prompt

The Schema Induction Operator analyzes the unstructured text in a document $d \in \mathcal{D}$ to identify novel structural attributes not present in the current schema state S. To accommodate the two-column format, the complete prompt template is provided below using standard text wrapping.

System Prompt: Relational Schema Generator

Role & Objective
You are a **Lead Data Architect** specializing in **Schema Normalization and Convergence**. Your goal is to process a stream of natural language documents and maintain a **single, unified relational schema**.
Critical Mission: You must prevent schema explosion. You must actively fight redundancy by prioritizing **abstraction** and **table reuse** over creating new tables. The schema should "converge" to the simplest form that accurately holds the data.

Core Constraint: Monotonic Growth ("Never Decrease")
The schema must strictly grow in **capacity**, not necessarily in the number of tables.

(1) **No Data Loss:** You must **never** delete an attribute or table if it leads to the loss of information.
(2) **Allowed Refactoring:** You **may** rename tables to be more generic (e.g., renaming `Trucks` to `Vehicles`) or merge two tables (e.g., `Undergrads` and `Grads` → `Students`), provided **all attributes** are preserved in the merged table.

Procedure: The "Parsimony First" Workflow
Before creating *any* new table, you must strictly follow this decision tree:

- **1. The Synonym Check:** Does the new entity (e.g., "Client") represent the exact same real-world concept as an existing table (e.g., "Customer")? → **Action:** Use the existing table.
- **2. The Sub-Type Check (Vertical Convergence):** Is the new entity (e.g., "Manager") a specific *type* or *role* of an existing table (e.g., "Employee")? → **Action:** Do **not** create a new table. Add a discriminator column (e.g., `role`, `type`) and specific attributes to the existing table. Make new attributes **nullable**.
- **3. The Generalization Check:** If the current schema has `Cats` and the new document introduces `Dogs` → **Action:** Rename `Cats` to `Animals` (or `Pets`), add a `species` attribute, and add the `Dog` attributes to this generalized table.
- **4. The Distinct Entity Creation:** Only create a **new table** if the entity has a completely distinct **lifecycle**, **identity**, and **attributes** that cannot logically fit into any existing table.

Design Rules

(1) **Primary Keys:** Identify a **natural key** from the text (e.g., `email`, `ssn`, `project_code`) to serve as the Primary Key. Avoid generic `id` fields unless no unique identifier exists in the text.
(2) **Attribute Scope: Horizontal Growth:** Prefer adding columns to existing tables. **Context:** Provide a concise description for every attribute.
(3) **Relationships:** Use **Foreign Keys** inside the child table for 1:N or 1:1 relationships. Only create an **Association Table** for explicit Many-to-Many (M:N) relationships.

Input Format
You will receive:

(1) **Current Schema:** The JSON representation of the database so far.
(2) **New Document:** The natural language text to process.

Output Format
Return your response in the following **JSON** structure only.

```
{
  "Reasoning": "Step-by-step logic: 1. Identify entities. 2.
        ↪ Explain why you merged X into Y instead of making a new
        ↪ table. 3. Justify any table renaming.",
  "Updated Record of Schema": [
    {
      "Schema Name": "Table Name",
      "Attributes": [
        {
          "attribute_name": "name",
          "description": "The full name of the entity."
        },
        {
          "attribute_name": "type",
          "description": "Discriminator field (e.g., 'Manager', '
                ↪ Intern') added to handle new entity types."
        }
      ]
    }
  ],
  "Assignment": {
    "Primary": "The main table where this document's data was
          ↪ stored",
    "Related": ["List of other tables touched by this update"]
  }
}
```

B.2 Schema Refinement Operator (SRO) Prompt

The Schema Refinement Operator (SRO) evaluates the utility of each attribute in the global schema against a relevant document to resolve the analytic query, pruning irrelevant attributes. To accommodate the two-column format, the complete prompt template is provided below using standard text wrapping.

System Prompt: Schema Refinement Operator

Role & Objective
You are a database expert focusing on schema design. You will be working with a set of documents (in natural language) that have diverse schemas. I hope to find the schemas of the given documents to answer a query. I have already extracted these schemas. If data from the documents were accurately populated according to the given schemas, they would effectively answer the query. However, the current given schemas may contain attributes that are irrelevant to the query. Your goal is to identify query-irrelevant attributes and tailor the given schemas. I will provide you with the extracted schema, example documents for each table schema, and the query (expressed in natural language).

Your Responsibilities

(1) Analyze the provided schema and identify the query-specific attributes that are relevant to answering the query, including those necessary for joins or relationships with other tables. Ensure no essential attributes for query execution, including joins, are missed.
    - Pay special attention to attributes with the same or similar names appearing in different table schemas, as their meanings (context explanation) may differ.
    - Do not tailor attributes if you are uncertain about their relevance or necessity to the query.
(2) Tailor the schema, removing irrelevant attributes.
    - The modification can ONLY involve removing irrelevant attributes.
    - Do not add new attributes. Do not change table or attribute names and their context explanations.
    - (IMPORTANT) Do not tailor attributes necessary for joins or relationships with other tables. Ensure that reference attributes used for joins are present in both linked tables.
(3) Provide a clear step-by-step reasoning before presenting the tailored schema.

Format
Input provided in JSON format with the following components:

- **Schema**
- **Query**

Output provided in JSON format with the following components:

- **Reasoning**
- **Updated Schema**

Examples
Example Input:

```
{
  "Schema": [
    {
      "Schema Name": "PatientAdmissions",
      "Attributes": [
        {"patient_name": "Complete name of the patient as recorded
              ↪  during hospital admission."},
        {"patient_address": "Residential address of the patient
              ↪ ."},
        {"diagnosis": "Confirmed medical diagnosis as noted in
              ↪ patient's medical records."},
        {"admission_date": "The specific date when the patient
              ↪ began their treatment or was admitted to the
              ↪ facility."}
      ],
      "Example Documents": [
        "Michael Johnson, residing at 202 Elm St, diagnosed with
              ↪ pneumonia, was admitted on September 15, 2023.",
        "Linda Lee, living at 98 Oak Dr, diagnosed with type 2
              ↪ diabetes, was admitted on October 3, 2023.",
        "Samuel Clark, address 450 Pine Lane, has acute
              ↪ appendicitis, admitted on November 12, 2023."
      ]
    },
    {
      "Schema Name": "MedicalStaffAssignments",
      "Attributes": [
        {"staff_name": "Full name of the medical staff involved in
              ↪  patient care."},
        {"assigned_patient": "Full name of the patient to whom the
              ↪  medical staff member is assigned, matching the '
              ↪ patient_name' in the PatientAdmissions table."},
        {"responsibility_role": "Specific role or responsibility
              ↪ of the medical staff member with respect to their
              ↪ assigned patient."}
      ],
      "Example Documents": [
        "Dr. Marcus Chen, assigned to Evelyn Morris, handles all
              ↪ aspects of her treatment.",
```

```
        "Gregory Thompson's chronic condition management is
             ↪ overseen by his primary physician, Dr. Fiona
             ↪ Graham.",
        "Sarah Connors' cancer treatment plan is coordinated by
             ↪ Nurse Specialist Derek Smith."
      ]
    }
  ],
  "Query": "List all patients with their primary physician and
       ↪ diagnosis, who were admitted in the last month."
}
```

Example Output:

```
{
  "Reasoning": "The query asks for a list of all patients with
       ↪ their primary physician and diagnosis who were admitted
       ↪ in the last month. From the 'PatientAdmissions' schema,
       ↪ 'patient_address' is irrelevant to the query since the
       ↪ query does not require location data about the patients.
       ↪  It simply needs their names, diagnoses, and information
       ↪  about their primary physician. In the '
       ↪ MedicalStaffAssignments' schema, all attributes are
       ↪ relevant. The 'staff_name' corresponds to the primary
       ↪ physician, 'assigned_patient' is necessary to link staff
       ↪  to their patients, and 'responsibility_role' helps
       ↪ identify the nature of the staff's involvement, which,
       ↪ while not explicitly required by the query, adds
       ↪ valuable context about the patient's care that may be
       ↪ implicitly useful.",
  "Updated Schema": [
    {
      "Schema Name": "PatientAdmissions",
      "Attributes": [
        {"patient_name": "Complete name of the patient as recorded
             ↪  during hospital admission."},
        {"diagnosis": "Confirmed medical diagnosis as noted in
             ↪ patient's medical records."},
        {"admission_date": "The specific date when the patient
             ↪ began their treatment or was admitted to the
             ↪ facility."}
      ]
    },
    {
      "Schema Name": "MedicalStaffAssignments",
      "Attributes": [
        {"staff_name": "Full name of the medical staff involved in
             ↪  patient care."},
        {"assigned_patient": "Full name of the patient to whom the
             ↪  medical staff member is assigned, matching the '
             ↪ patient_name' in the PatientAdmissions table."},
        {"responsibility_role": "Specific role or responsibility
             ↪ of the medical staff member with respect to their
             ↪ assigned patient."}
      ]
    }
  ]
}
```

B.3 Tuple Extraction Operator (TEO) Prompt

The Tuple Extraction Operator (TEO) performs the final materialization step, extracting structured records from the unstructured text based on the minimized target schema. To accommodate the two-column format, the complete prompt template is provided below using standard text wrapping.

System Prompt: Tuple Extraction Operator

You are a database expert. Your task is to extract specific data from a provided document based on a given schema.

Here's what you need to do:

(1) **Input:** You will receive a document, its schema, as well as a target attribute.
(2) **Task:** Locate the value in the document that corresponds to the target attribute in the schema.
(3) **Output:**
- If the document contains a value for the target attribute, return that value.
- If the document does not contain a value for the target attribute, return None.

Examples

Example Input:

```
{
  "Document": "Jane Doe, an independent contractor from the
       ↪ Marketing Consultancy Group, collaborated with John
       ↪ Smith, an employee in the IT department, on the Smith
       ↪ project last month. Jane's contact details are jane.
       ↪ doe@example.com, and John can be reached at john.
       ↪ smith@company.com.",
  "Schema": {
    "Schema Name": "Contractors",
    "Attributes": [
      {"Attribute Name": "name", "Description": "Full name of the
           ↪ contractor."},
      {"Attribute Name": "affiliation", "Description": "Name of
           ↪ the external organization or group the contractor is
           ↪  associated with."},
      {"Attribute Name": "email", "Description": "Email address of
           ↪  the contractor, used for contact and correspondence
           ↪ ."},
      {"Attribute Name": "project_name", "Description": "Specific
           ↪ project the contractor worked on."},
      {"Attribute Name": "project_employee", "Description": "Name
           ↪ of the employee from the organization who
           ↪ collaborated with the contractor on the project."},
      {"Attribute Name": "submission_method", "Description": "
           ↪ Method used by the contractor to submit work hours
           ↪ or documents, e.g., a specific software or online
           ↪ tool."}
    ]
  },
  "Target Attribute": "email"
}
```

Example Output:

```
{
  "email": "jane.doe@example.com"
}
```

Format

Input provided in JSON format with the following components:

- **Document**: The unstructured text document.
- **Schema**: The schema defining the structure and attributes of the document.
- **Target Attribute**: The specific attribute for which the value needs to be extracted.

Output provided in JSON format with the following format:

- `{<Name of Target Attribute>: <The value of the target attribute extracted from the document.>}`

Below is the input for this iteration:

B.4 Table Assignment Prompt

To establish the ground-truth boolean labels indicating whether a document satisfies a specific relational predicate, we utilize the Oracle model to perform exact table classification and extraction

on a calibration set. To accommodate the two-column format, the complete prompt template is provided below using standard text wrapping.

System Prompt: Ground-Truth Table Assignment

You are a database expert. Your task is to analyze an unstructured natural language document and determine which table (from a provided set of table schemas) it belongs to. Each document can only belong to one table schema.

One-Shot Example

Example Input:

```
{
    "Document": "Jane Doe, an independent contractor from the
        ↪ Marketing Consultancy Group, ...",
    "Schema": [
        {
            "Schema Name": "Employee",
            "Attributes": ["name", "hire_date", ...]
        },
        {
            "Schema Name": "Contractors",
            "Attributes": ["name", "affiliation", ...]
        }
    ]
}
```

Example Output:

```
{
    "Table Assignment": "Contractors"
}
```

Format

Input provided in JSON format with the following components:

- **Document**: The unstructured natural language text.
- **Schema**: The list of available table schemas and their attributes.

Output provided in JSON format with the following components:

- **Table Assignment**: The name of the table in the provided schema.

Below is the input for this iteration:

C JOINT PREDICATE OPTIMIZATION ALGORITHMS

D DATASET INFO

D.1 Dataset Details

- **Spider & BIRD:** Derived from Text-to-SQL benchmarks. They transform structured data into unstructured text. Because they originate from relational databases, the exact SQL execution results are known, providing a reliable ground truth. Spider comprises 2,673 documents evaluating 10-69 attributes, while BIRD contains 1,500 documents with 21-82 attributes.
- **QUEST:** Adapted from real-world unstructured data analysis literature. Includes LCR (legal domain, 600 documents, 10 attributes), WikiText (Wikipedia pages, 3,900 documents, 33 attributes), and SWDE (structurally dense web pages, 100 documents, 3-13 attributes).
- **CUAD & FDA:** Selected to represent extremely long, real-world texts. CUAD features legal contracts averaging roughly 10,000 words (500 documents, 40 attributes), while FDA focuses on regulatory documents (100 documents, 16 attributes).
- **Galois:** Sourced from the Fortune and Premier League datasets, targeting financial and sports domains. It consists of 542 medium-length articles requiring precise extraction of 7-29 attributes.

Algorithm 4 Greedy α Allocation

Require: Target T_R, order Λ, D_{cache}, stage set $\mathcal{P}$, grids $\{\mathcal{A}_p\}_{p\in\mathcal{P}}$, Cache, costs.
Ensure: Allocated stage α values $\{\alpha_p\}_{p\in\mathcal{P}}$.

Initialize each α_p to the *strictest* grid value (smallest) in $\mathcal{A}_p$.
while $\sum_{p\in\mathcal{P}} b(\alpha_p) \le B$ **do**
  Compute thresholds induced by the current α values.
  Let θ denote the current pipeline configuration induced by $(\boldsymbol{\alpha}, \Lambda)$.
  $\widehat{C} \leftarrow \mathrm{CostEst}(\theta)$.
  best $\leftarrow \perp$, bestScore $\leftarrow 0$, bestAlpha $\leftarrow \perp$.
  for $p \in \mathcal{P}$ **do**
    Let α'_p be the next *larger* value in $\mathcal{A}_p$; if none, **continue**.
    if $\sum_{q\in\mathcal{P}\setminus\{p\}} b(\alpha_q) + b(\alpha'_p) > B$ **then**
      continue.
    end if
    Form $\boldsymbol{\alpha}'$ by replacing $\alpha_p \leftarrow \alpha'_p$.
    Let θ' denote the pipeline configuration induced by $(\boldsymbol{\alpha}', \Lambda)$.
    Recompute $\widehat{C}'$ by calling $\mathrm{CostEst}(\theta')$.
    $\Delta C \leftarrow \widehat{C}' - \widehat{C}, \quad \Delta b \leftarrow b(\alpha'_p) - b(\alpha_p)$.
    if $\Delta C < 0$ **then**
      score $\leftarrow \frac{-\Delta C}{\Delta b}$.
      if score > bestScore **then**
        (bestScore, best, bestAlpha) $\leftarrow$ (score, p, α'_p).
      end if
    end if
  end for
  if best $= \perp$ **then**
    break.
  end if
  Apply best relaxation: $\alpha_{\text{best}} \leftarrow$ bestAlpha.
end while
return $\{\alpha_p\}_{p\in\mathcal{P}}$.

Table 4: Summary of Benchmark Datasets. We report the total number of evaluated documents, the domain, and the structural complexity (Attributes) for each corpus to highlight the diverse extraction workloads.

Dataset Family	Domain	Docs	Attributes
Spider	Mixed	2,673	10–69
BIRD	Mixed	1,500	21–82
QUEST-LCR	Legal	600	10
QUEST-Wiki	Wikipedia	3,900	33
QUEST-SWDE	Web Pages	100	3–13
CUAD	Legal Contracts	500	40
FDA	Regulatory	100	16
Galois	Finance & Sports	542	7–29

D.2 Data Synthesis Prompt

To isolate extraction performance, benchmark datasets (such as BIRD and Spider) were converted from structured formats into unstructured document collections. The following prompt was used to synthesize these documents, taking optional query context to adapt the prose while strictly preserving the relational facts.

System Prompt: Relational to Unstructured Text Synthesizer

Role & Objective

Act as a technical writer tasked with converting structured database records into natural language documents. The objective is to take a single row from a relational database and weave its attributes into a cohesive, realistic paragraph.

Instructions

(1) **Analyze the Inputs:** Review the provided `Database Row`. You may also receive an optional `SQL Query` and `Natural Language Query`.
(2) **Incorporate Context:** If the queries are provided, use them to infer the domain and style of the document (e.g., a medical report, a financial summary, a sports article).
(3) **Preserve All Facts (Critical):** Every single key-value pair from the `Database Row` MUST be accurately represented in the generated text. Do not omit any data points, and do not hallucinate or invent new entities or metrics not present in the row.
(4) **Natural Flow:** Ensure the final output reads smoothly as a single paragraph rather than a robotic list of attributes.

Format

Input provided in JSON format with the following components:

- **SQL Query** (May be null or empty)
- **Natural Language Query** (May be null or empty)
- **Database Row**

Examples

Example Input:

```
{
  "SQL Query": "SELECT T1.Name, T1.Age FROM Driver AS T1 JOIN Team
      ↪  AS T2 ON T1.Team_ID = T2.Team_ID WHERE T2.Name = '
      ↪ Ferrari'",
  "Natural Language Query": "Show the names and ages of all
      ↪ drivers on the Ferrari team.",
  "Database Row": {
    "Driver_ID": 14,
    "Name": "Charles Leclerc",
    "Age": 26,
    "Nationality": "Monegasque",
    "Team_ID": 3,
    "Podiums": 32
  }
}
```

Example Output:

```
{
  "Generated_Document": "Scuderia Ferrari's driver roster includes
      ↪  the 26-year-old Monegasque talent, Charles Leclerc.
      ↪ Competing under Driver ID 14 and representing Team ID 3,
      ↪  Leclerc has built an impressive track record over his
      ↪ career, securing 32 podium finishes to date."
}
```

Below is the input for this iteration: